Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
Уральский технический институт связи и информатики (филиал) в г. Екатеринбурге
ЦК ИТ и АСУ кафедры ИСТ

Отчет
по учебной практике
по МДК 02.02: «Инструментальные средства разработки программного обеспечения»
на тему: «Создание программного комплекса для видеомониторинга и управления доступом сотрудников и автотранспорта»

студента IV курса 283 группы

Фамилия Патрушев

Имя, отчество Антон Александрович

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы и программирование»

г. Екатеринбург 2026 г.

Отзыв руководителя

Содержание

Введение 4
1 Выполнение задания демонстрационного экзамена 5
1.1 Анализ предметной области 5
1.2 Аналогичные программные решения 5
1.3 Требования к программному продукту 5
1.4 Обоснование выбора инструментальных средств разработки 7
1.5 Проектирование и разработка базы данных 7
1.6 Разработка серверного API 10
1.7 Разработка пользовательского интерфейса 11
1.8 Тестирование приложения 20
2 Практическая реализация дипломного проекта 22
2.1 Общее описание системы и задачи ИИ-модуля 22
2.2 Выбор инструментов и обоснование решений 23
2.3 Подготовка и описание датасетов 24
2.4 Обучение модели детекции YOLOv11m 25
2.5 Результаты детекции 27
2.6 Разработка модуля оптического распознавания символов 29
2.7 Единый модуль обработки кадра 32
2.8 Схема базы данных 34
2.9 Безопасность и разграничение прав доступа 37
2.10 Результаты и направления развития 39
Заключение 40
Библиография 41
Приложение А 42

Введение

За время учебной практики было сделано два направления работы: сначала выполнено задание пробного демонстрационного экзамена, а затем продолжена работа по дипломному проекту.

По заданию пробного ДЭ нужно было разработать систему «АвтоТранс» для учёта заявок на ремонт автомобилей. В системе четыре роли: клиент (заказчик), автомеханик, менеджер по качеству и администратор.

В качестве технологического стека применялись: C# с фреймворком ASP.NET Core 10.0 на серверной части, PostgreSQL 15 в качестве СУБД, JavaScript/HTML/CSS для клиентского интерфейса, Postman 11 для тестирования API.

Цели выполнения задания:

– спроектировать реляционную базу данных для учёта заявок на ремонт автомобилей;

– реализовать серверный REST API с ролевым разграничением доступа на cookie;

– разработать клиентский веб-интерфейс для всех четырёх ролей;

– провести функциональное тестирование.

1 Выполнение задания демонстрационного экзамена

1.1 Анализ предметной области

Предметная область – автосервис. Клиенты приводят машины, механики делают ремонт, ведётся учёт работ и расходников, менеджер смотрит статистику.

Основные участники бизнес-процесса:

– клиент (владелец автомобиля) – обращается в сервис, описывает неисправность, отслеживает статус ремонта;

– автомеханик – выполняет ремонтные работы, фиксирует ход работ, запчасти и результат;

– менеджер по качеству – распределяет заявки, контролирует сроки, ведёт коммуникацию, анализирует статистику;

– администратор – управляет учётными записями сотрудников системы.

Жизненный цикл заявки включает статусы: «Новая» (создана, механик не назначен), «В работе» (механик назначен), «Ожидание» (ожидание запчастей), «Завершено», «Отменено».

1.2 Аналогичные программные решения

На рынке присутствуют ERP/CRM-системы для автосервисов (1С: Автосервис, Битрикс24), требующие дорогостоящего внедрения, и мобильные приложения (Яндекс Услуги), не обеспечивающие полного цикла управления ремонтом. Разрабатываемый модуль «АвтоТранс» ориентирован на конкретные нужды малого автосервиса и содержит только необходимый функционал, что упрощает его внедрение и использование.

1.3 Требования к программному продукту

Функциональные требования к каждой роли:

– клиент – регистрация, создание, просмотр, редактирование и отмена заявок, отправка обращений в поддержку;

– менеджер по качеству – просмотр всех заявок, назначение механиков, смена статусов, ответы на обращения, просмотр статистики;

– автомеханик – просмотр назначенных заявок, добавление комментариев, фиксация запчастей, завершение ремонта;

– администратор – управление учётными записями сотрудников.

Нефункциональные требования: использование веб-интерфейса, разграничение доступа на основе ролей, хранение данных в реляционной СУБД.

Полный перечень функциональных требований систематизирован в таблице 1.1. Требования сгруппированы по ролям пользователей системы.

Таблица 1.1 – Функциональные требования к системе «АвтоТранс»

№	Роль	Требование
1	Клиент	Регистрация в системе по логину и паролю
2	Клиент	Создание заявки на ремонт с указанием вида, модели автомобиля и описания неисправности
3	Клиент	Просмотр списка своих заявок с фильтрацией по статусу
4	Клиент	Отмена заявки со статусом «Новая»
5	Клиент	Отправка обращений в службу поддержки и просмотр ответов
6	Менеджер	Просмотр всех заявок системы с поиском и фильтрацией
7	Менеджер	Назначение автомеханика на заявку с учётом его текущей нагрузки
8	Менеджер	Смена статусов заявок, просмотр KPI-статистики
9	Менеджер	Обработка входящих обращений клиентов, отправка ответов
10	Автомеханик	Просмотр только назначенных ему заявок
11	Автомеханик	Добавление комментариев к ходу ремонта, фиксация использованных запчастей
12	Автомеханик	Завершение ремонта и перевод заявки в статус «Завершено»
13	Администратор	Создание, редактирование и деактивация учётных записей сотрудников
14	Администратор	Назначение и изменение ролей сотрудников в системе

Нефункциональные требования зафиксированы отдельно. Система должна функционировать в браузерном окружении без установки дополнительного программного обеспечения на стороне клиента. Разграничение доступа реализовано на уровне бизнес-логики сервера и на уровне СУБД через роли PostgreSQL. Все данные хранятся в единой реляционной СУБД PostgreSQL 15 со средствами резервного копирования.

1.4 Обоснование выбора инструментальных средств разработки

– ASP.NET Core 10.0 – кроссплатформенный фреймворк для веб-приложений на C#. Шаблон MVC позволяет разделить бизнес-логику, представление и обработку запросов. Встроенная cookie-аутентификация упрощает управление сессиями.

– PostgreSQL 15 – объектно-реляционная СУБД для хранения данных. Надёжная, производительная, с поддержкой ролевого разграничения доступа на уровне СУБД.

– Entity Framework Core – ORM для работы с базой данных через типизированные C#-объекты, ускоряет разработку.

– Visual Studio 2022 – интегрированная среда разработки.

– Postman 11 – инструмент для тестирования HTTP API.

– draw.io – веб-инструмент для построения ER-диаграмм и блок-схем.

– GitHub – облачный репозиторий для хранения истории разработки.

1.5 Проектирование и разработка базы данных

На этапе анализа предметной области выделены четыре основные сущности: «Пользователь» (объединяет клиентов и сотрудников), «Заявка», «Комментарий» и «Сообщение». Единая таблица пользователей выбрана намеренно: поле Type определяет роль записи.

Связи между сущностями:

– Users – Requests (1:N по ClientID и по MasterID);

– Requests – Comments (1:N);

– Users – Comments (1:N);

– Users – Messages (1:N).

ER-диаграмма базы данных представлена на рисунке 1.1, физическая схема – на рисунке 1.2.

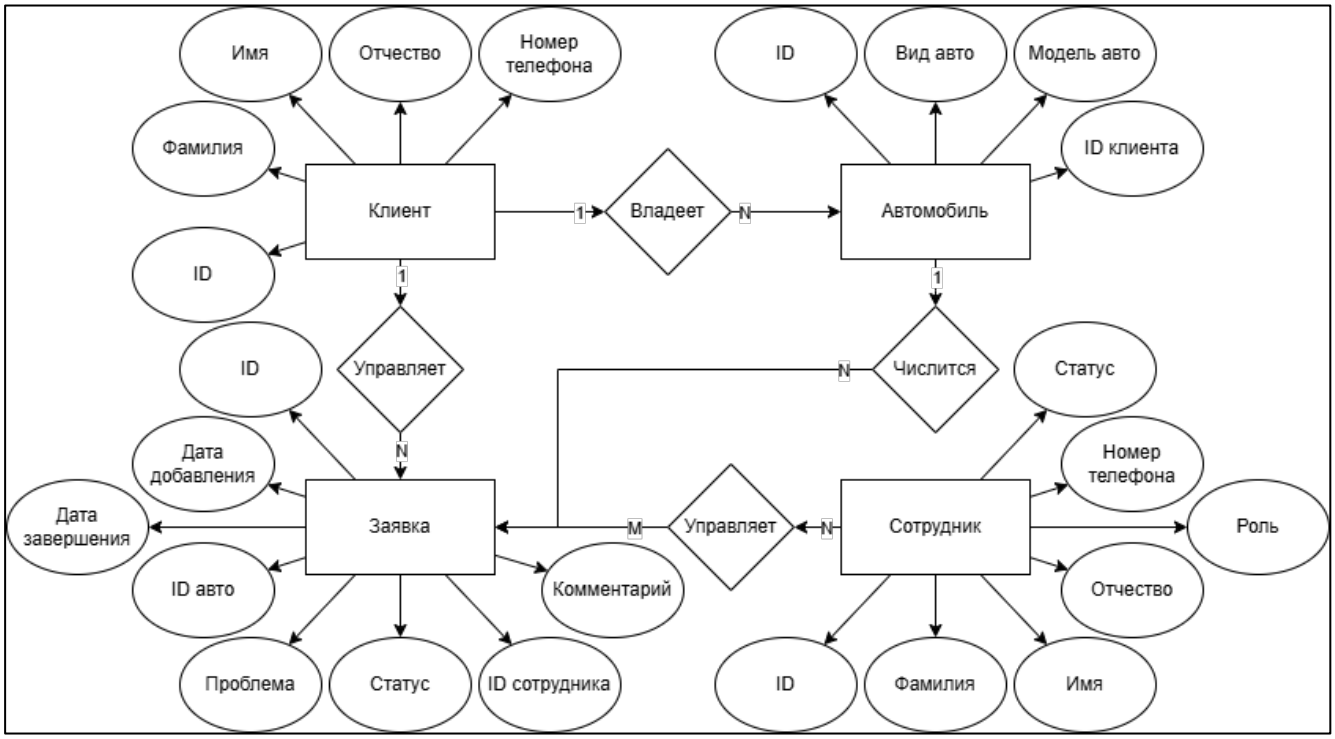

Рисунок 1.1 – ER-диаграмма базы данных

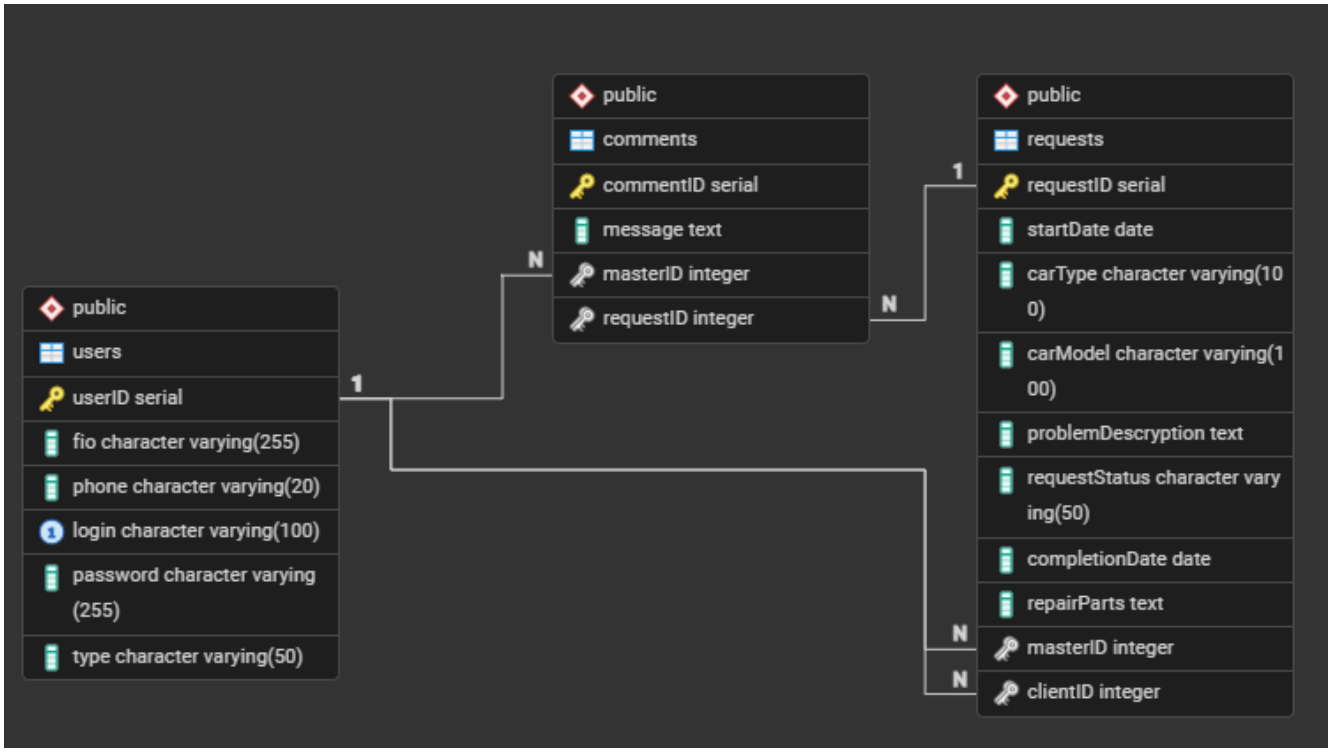

Рисунок 1.2 – Физическая схема базы данных

SQL-код создания структуры базы данных приведён в листинге 1.1.

Листинг 1.1 – SQL-код создания таблиц базы данных

```
CREATE TABLE "Users" (
    "UserID"   SERIAL        PRIMARY KEY,
    "Fio"      VARCHAR(255)  NOT NULL,
    "Phone"    VARCHAR(20)   NOT NULL,
    "Login"    VARCHAR(100)  UNIQUE NOT NULL,
    "Password" VARCHAR(255)  NOT NULL,
    "Type"     VARCHAR(50)   NOT NULL
);
CREATE TABLE "Requests" (
    "RequestID"          SERIAL        PRIMARY KEY,
    "StartDate"          DATE          NOT NULL,
    "CarType"            VARCHAR(100)  NOT NULL,
    "CarModel"           VARCHAR(100)  NOT NULL,
    "ProblemDescryption" TEXT          NOT NULL,
    "RequestStatus"      VARCHAR(50)   NOT NULL,
    "CompletionDate"     DATE,
    "RepairParts"        TEXT,
    "MasterID"  INTEGER REFERENCES "Users"("UserID") ON DELETE SET NULL,
    "ClientID"  INTEGER NOT NULL
                REFERENCES "Users"("UserID") ON DELETE CASCADE
);
CREATE TABLE "Comments" (
    "CommentID" SERIAL  PRIMARY KEY,
    "Message"   TEXT    NOT NULL,
    "MasterID"  INTEGER NOT NULL REFERENCES "Users"("UserID") ON DELETE CASCADE,
    "RequestID"  INTEGER  NOT  NULL  REFERENCES  "Requests"("RequestID")  ON  DELETE
CASCADE
);
CREATE TABLE "Messages" (
    "MessageID"   SERIAL        PRIMARY KEY,
    "Subject"     VARCHAR(255)  NOT NULL,
    "MessageText" TEXT          NOT NULL,
    "ClientID"    INTEGER NOT NULL REFERENCES "Users"("UserID") ON DELETE CASCADE,
    "Reply"       TEXT,
    "IsRead"      BOOLEAN NOT NULL DEFAULT FALSE,
    "SentAt"      DATE
);
```

Разграничение доступа на уровне СУБД реализовано через роли PostgreSQL (Листинг 1.2).

Листинг 1.2 – SQL-код ролей и привилегий

```
CREATE ROLE client_role;   CREATE ROLE mechanic_role;
CREATE ROLE manager_role;  CREATE ROLE admin_role;
GRANT   SELECT   ON   "Requests","Users","Comments","Messages"   TO
client_role;
GRANT INSERT ON "Requests","Messages" TO client_role;
GRANT SELECT, UPDATE ON "Requests","Comments" TO mechanic_role;
GRANT INSERT         ON "Comments"             TO mechanic_role;
GRANT SELECT, INSERT, UPDATE
    ON "Requests","Users","Comments","Messages" TO manager_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_role;
```

1.6 Разработка серверного API

Серверная часть разработана по шаблону Model-View-Controller. Каждая таблица обслуживается отдельным контроллером. Все реализуют CRUD через методы GET (список), GET/{id}, POST, PUT/{id}, DELETE/{id}.

Аутентификация реализована на основе cookie. При успешном входе создаётся ClaimsPrincipal с идентификатором пользователя, его ФИО и ролью, который зашифровывается в cookie. Каждый защищённый метод помечен атрибутом [Authorize]; при отсутствии cookie возвращается код 401.

Изоляция данных реализована на уровне бизнес-логики. При запросе GET /api/requests клиент получает только свои заявки, механик – только назначенные, менеджер и администратор – весь список.

Пример реализации метода GET с ролевой фильтрацией приведён в листинге 1.3.

Листинг 1.3 – Метод GET контроллера заявок

```
[Authorize]
[HttpGet]
public async Task<IActionResult> GetAll()
{
    var type   = User.FindFirst("Type")?.Value;
    var userId = int.Parse(User.FindFirst("UserID")?.Value ?? "0");
    var query  = _context.Requests
        .Include(r => r.Master).Include(r => r.Client)
        .AsQueryable();
    if (type == "заказчик")
        query = query.Where(r => r.ClientID == userId);
    else if (type == "автомеханик")
        query = query.Where(r => r.MasterID == userId);
    return Ok(await query.OrderByDescending(r => r.StartDate)
        .ToListAsync());
}
```

Проверка API в Postman показала корректную работу всех эндпоинтов. Результаты тестирования представлены на рисунках 1.3 и 1.4.

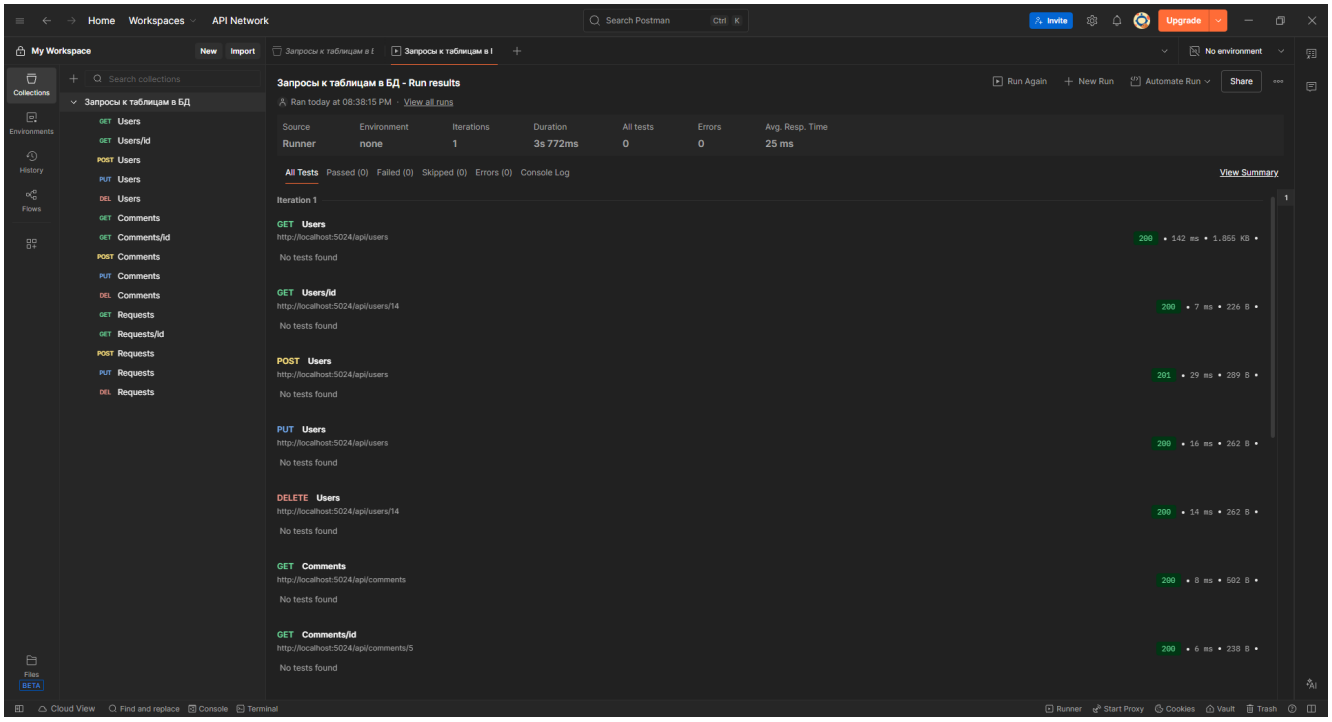

Рисунок 1.3 – Тестирование API Users и Comments в Postman

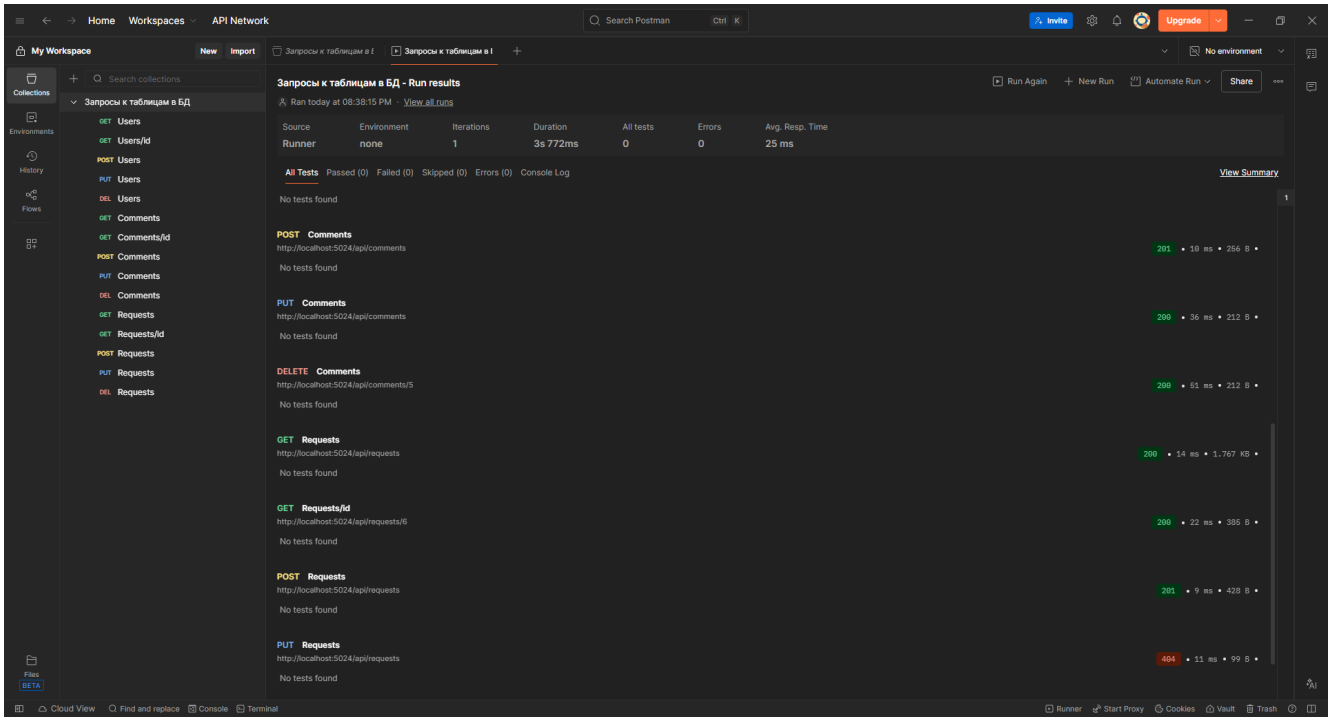

Рисунок 1.4 – Тестирование API Requests в Postman

1.7 Разработка пользовательского интерфейса

Клиентский интерфейс разработан на HTML5, CSS3 и JavaScript в составе Razor-представлений ASP.NET Core. Все страницы выполнены в единой бежево-коричневой гамме. Навигация – верхнее меню для клиентов, боковая панель для

сотрудников. На рисунках 1.5-1.17 представлены основные экранные формы приложения.

Страница входа (Рисунок 1.5) содержит форму с полями логин/пароль и вкладки для переключения между входом и регистрацией.

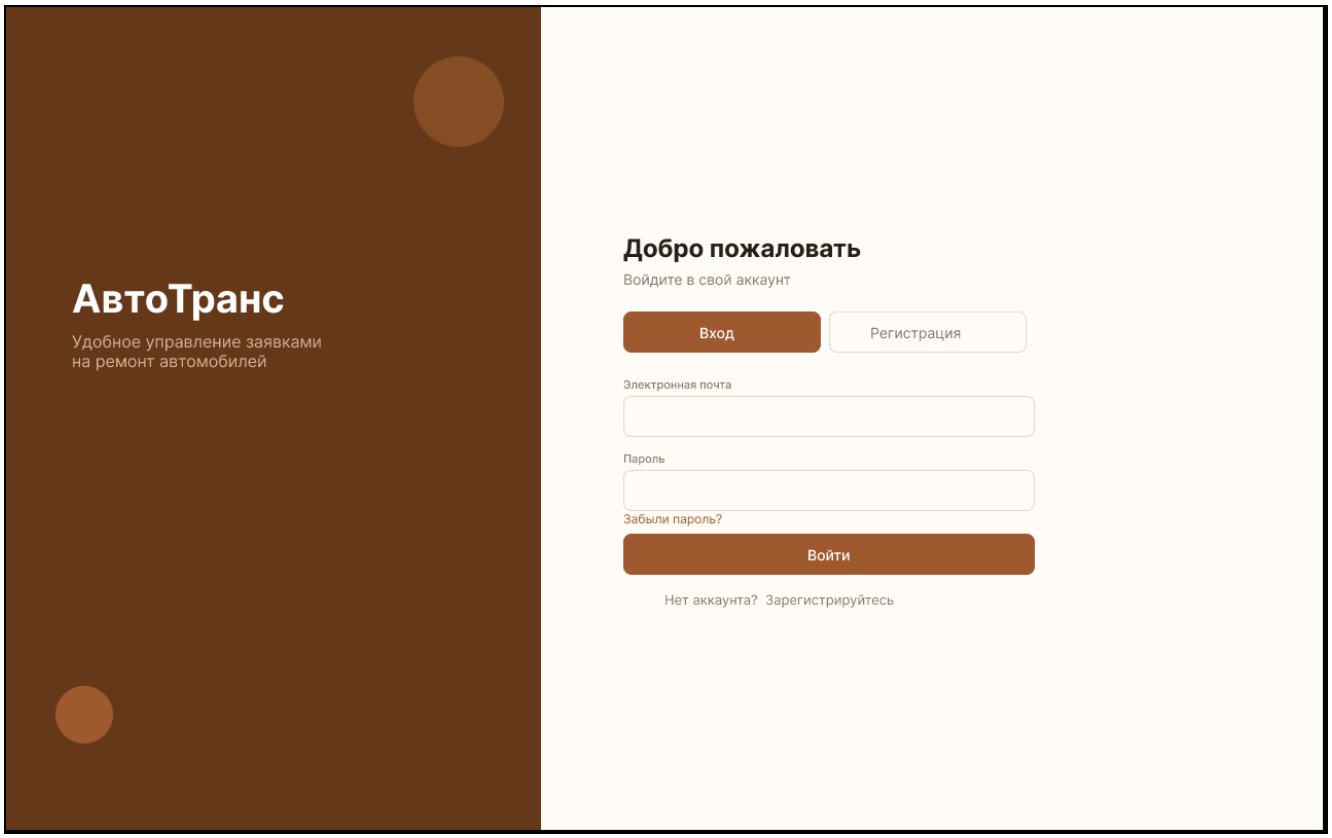

Рисунок 1.5 – Страница входа в систему

Личный кабинет клиента (Рисунок 1.6) отображает профиль, список автомобилей и сводную статистику по заявкам в виде карточек.

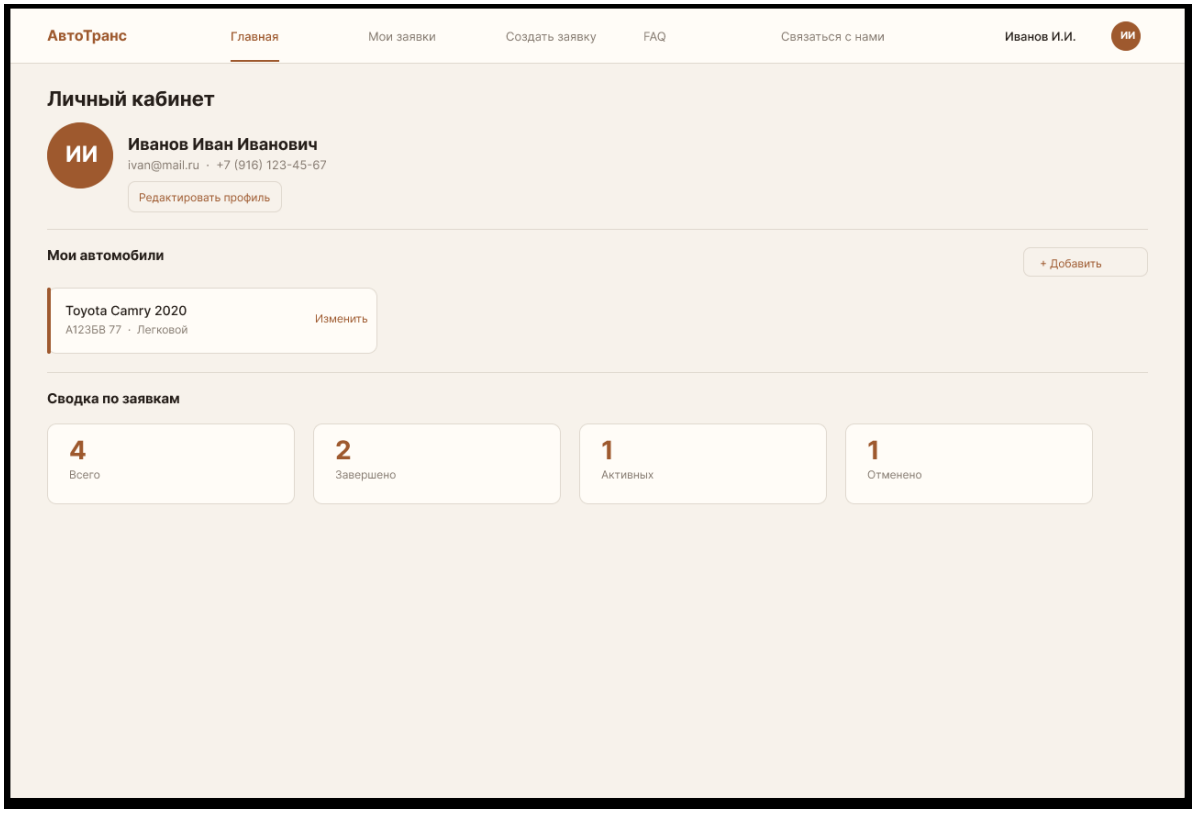

Рисунок 1.6 – Личный кабинет клиента

Форма создания заявки (Рисунок 1.7): вид и модель автомобиля, описание проблемы, желаемая дата. На правой панели – инструкция и QR-код оценки сервиса.

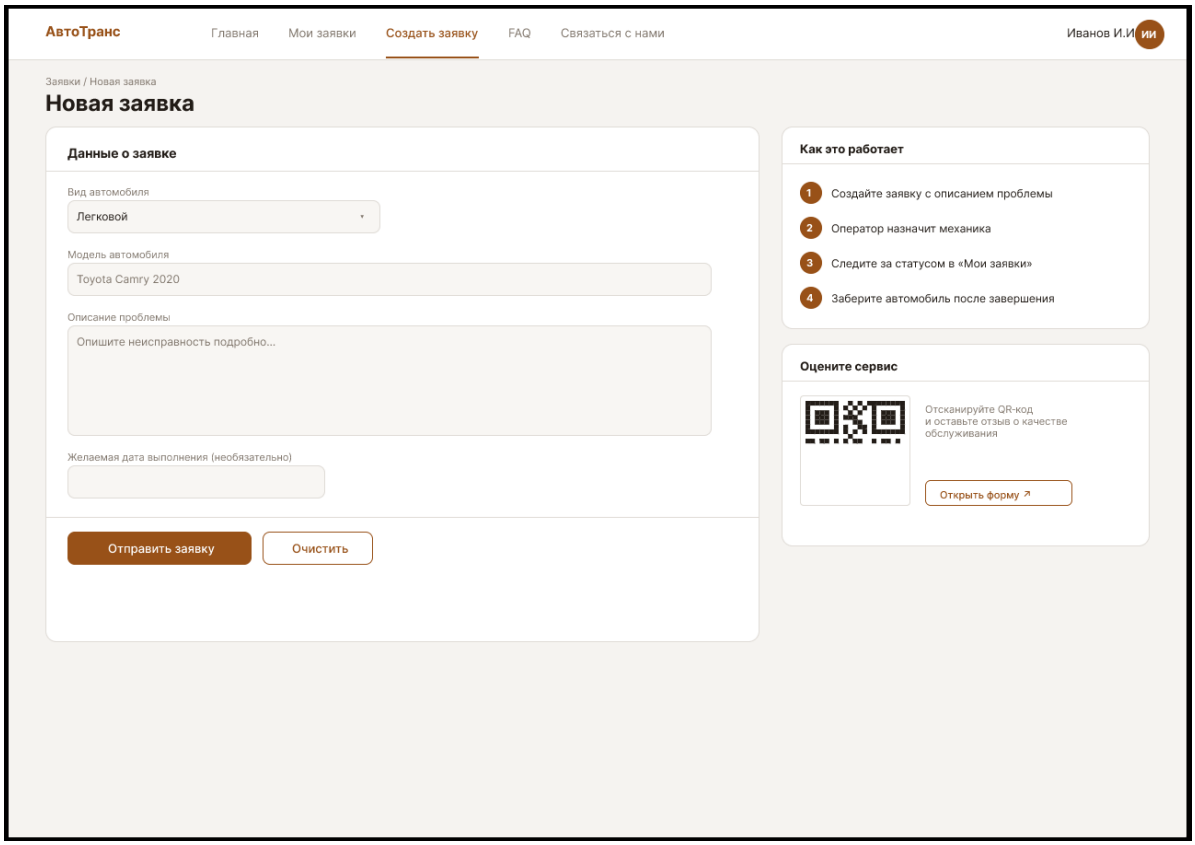

Рисунок 1.7 – Форма создания заявки

Страница «Мои заявки» (Рисунок 1.8): список заявок с фильтрацией по статусу. Карточка заявки (Рисунок 1.9): все поля, назначенный механик, история статусов, кнопка отмены (только для новой заявки).

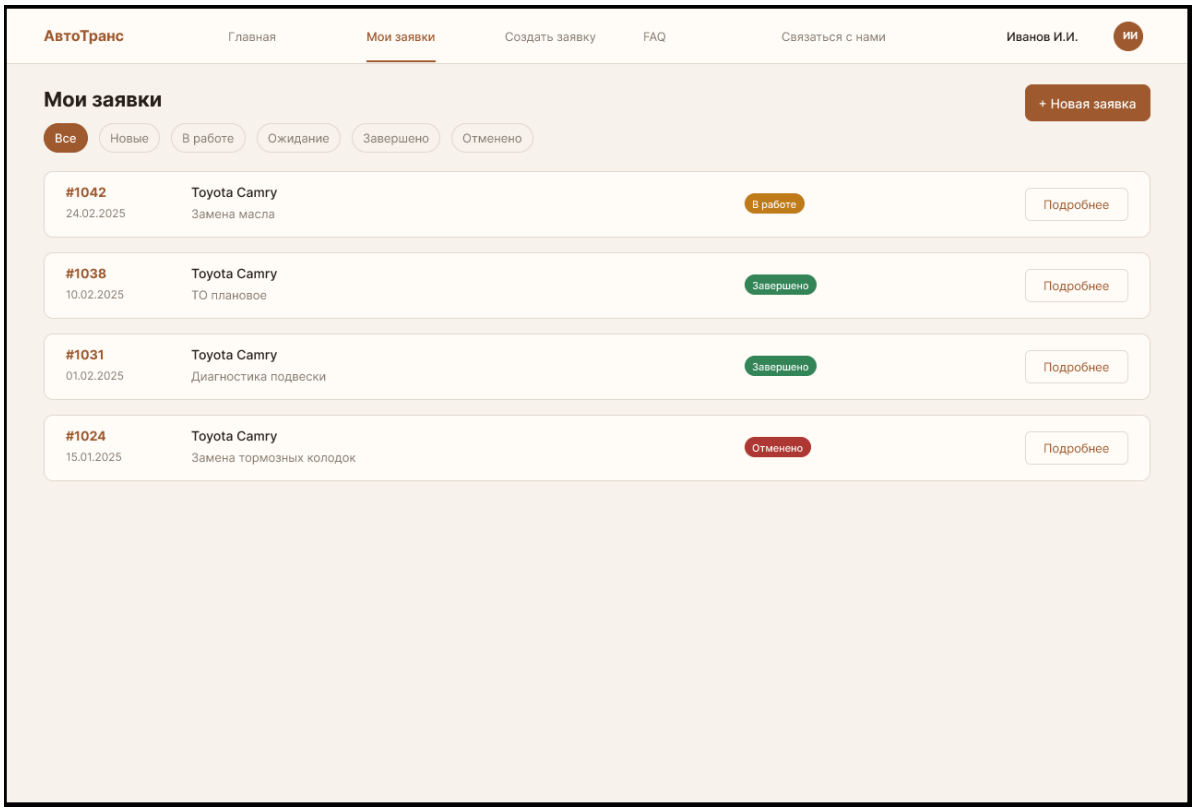

Рисунок 1.8 – Список заявок клиента

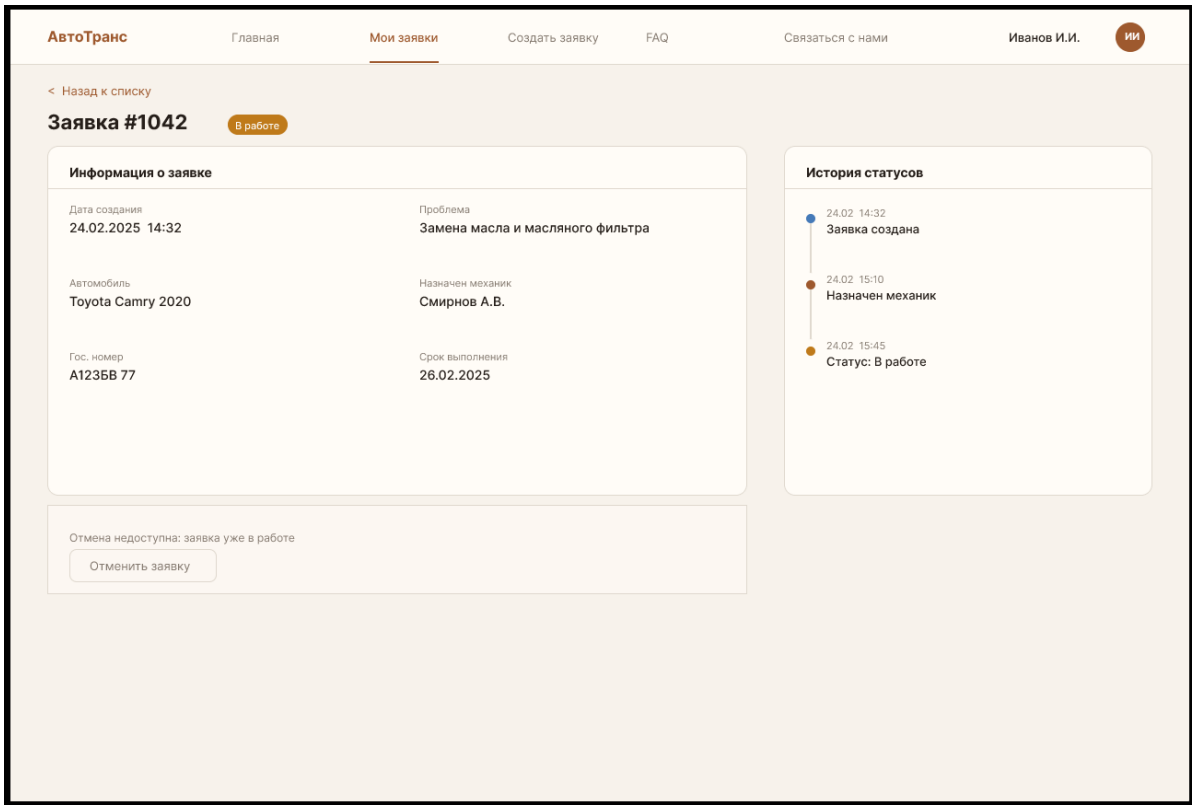

Рисунок 1.9 – Просмотр заявки клиентом

Страница «Связаться с нами» (Рисунок 1.10): форма обращения, контакты, QR-код Google Forms. Страница FAQ (Рисунок 1.11): аккордеонный список ответов с поиском.

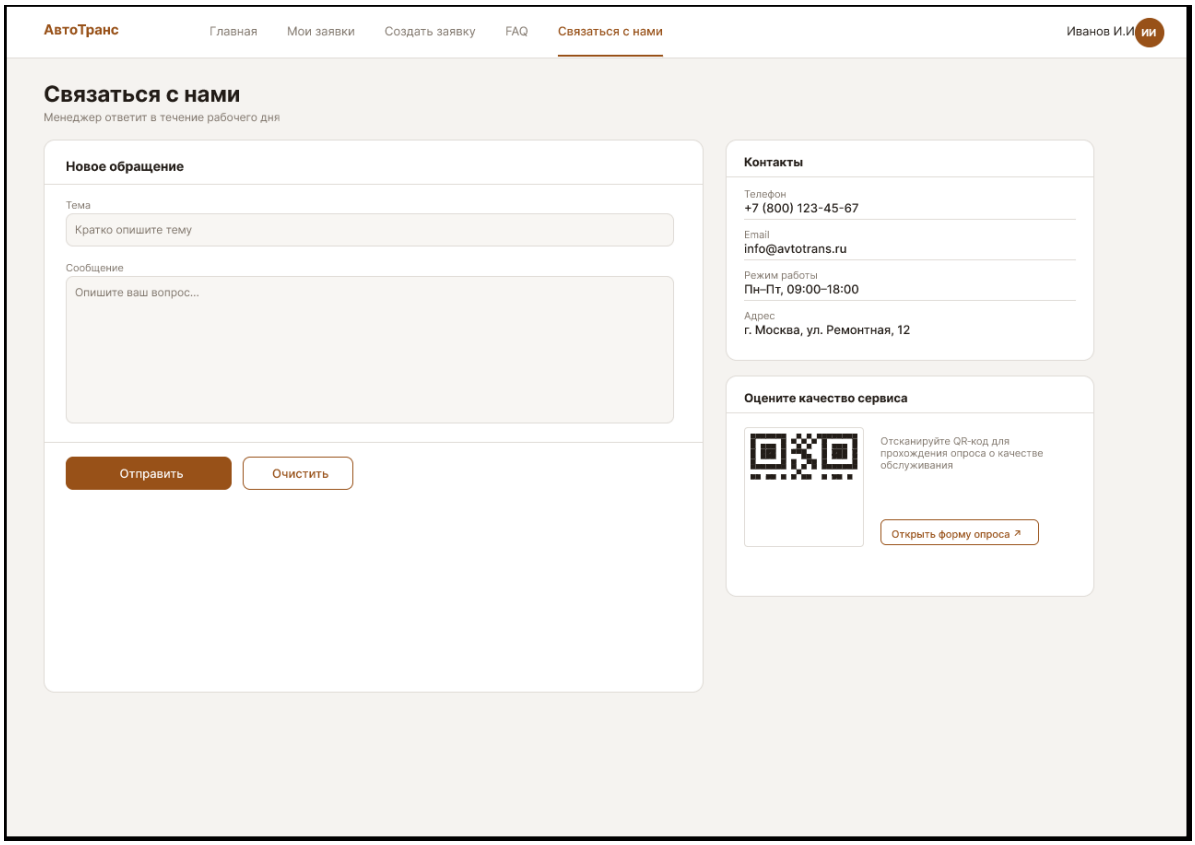

Рисунок 1.10 – Страница обратной связи

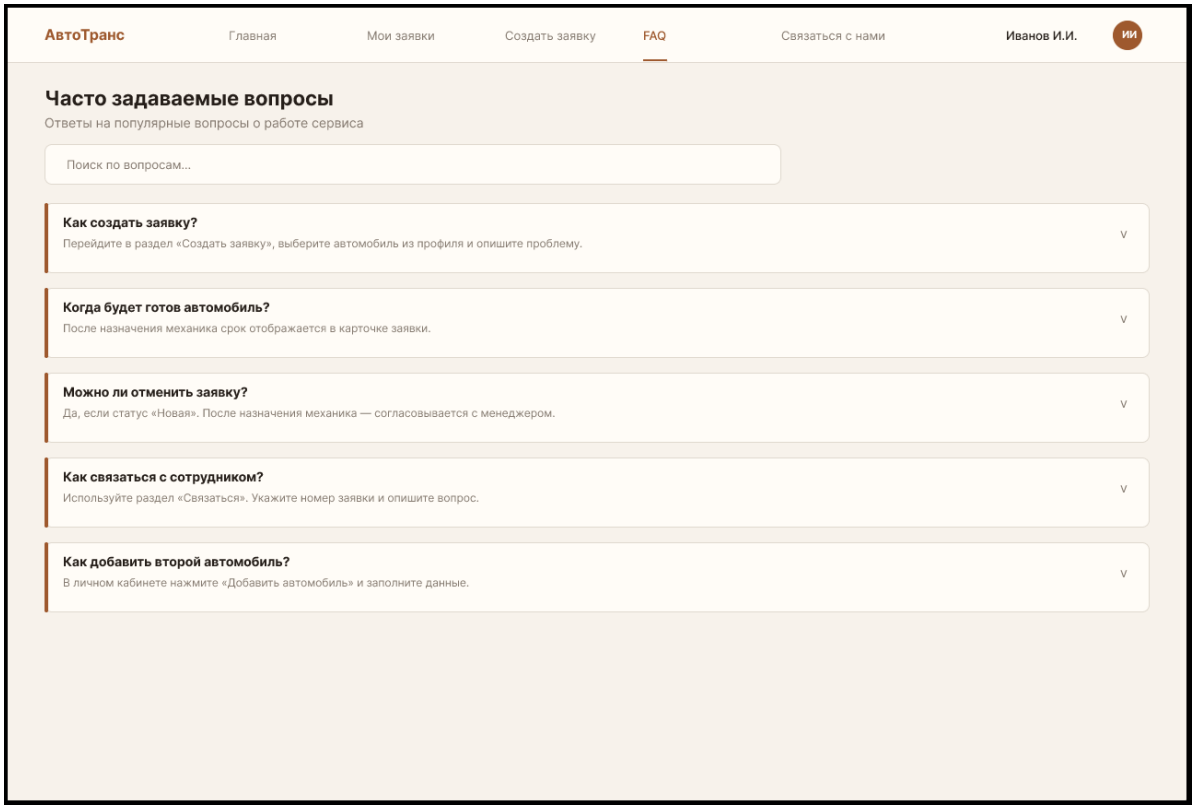

Рисунок 1.11 – Страница часто задаваемых вопросов

Интерфейс менеджера включает: список всех заявок с поиском (Рисунок 1.12), форму назначения механика с указанием нагрузки (Рисунок 1.13), страницу статистики с KPI-карточками и графиком (Рисунок 1.14), страницу сообщений с входящими обращениями клиентов (Рисунок 1.15).

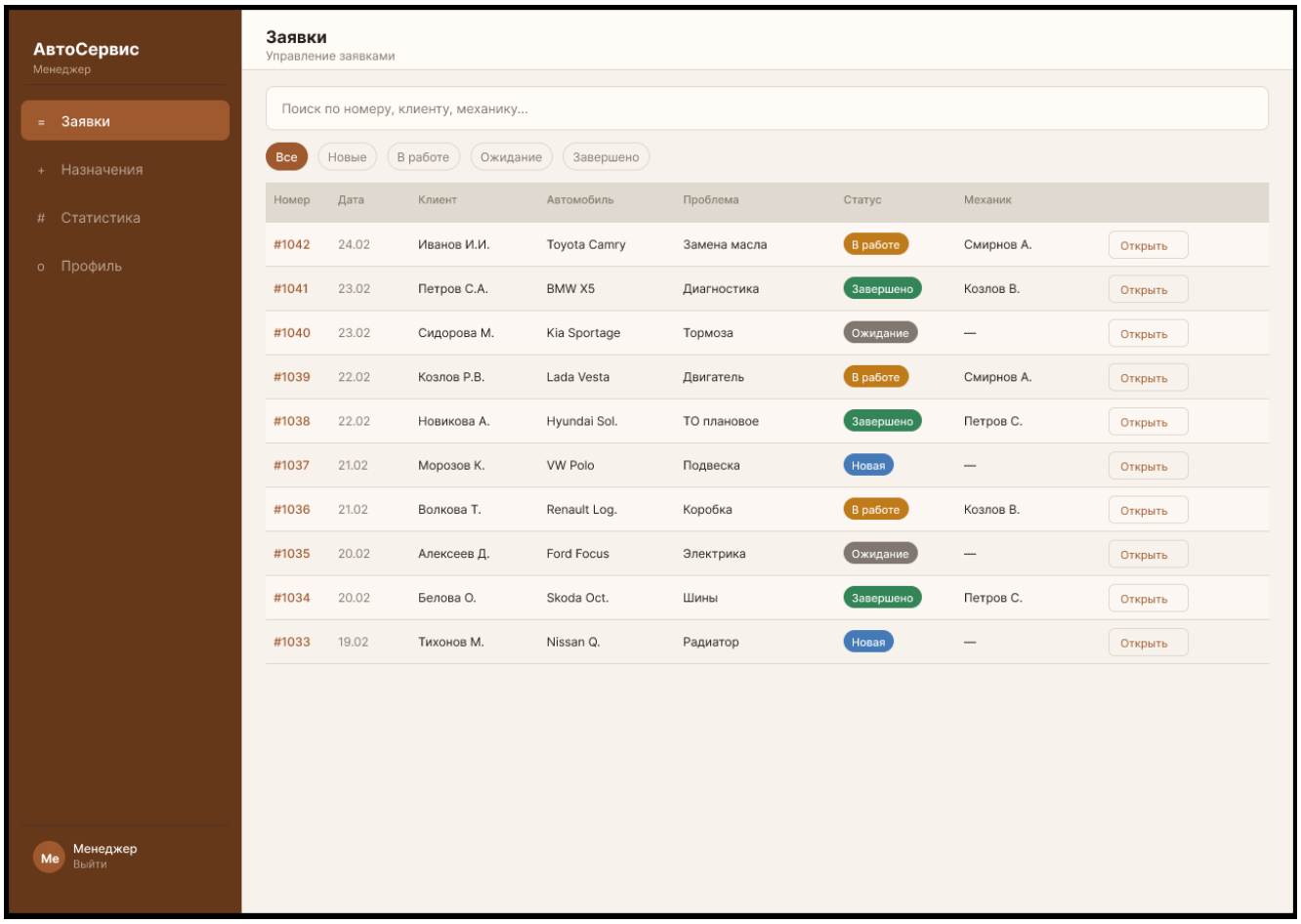

Рисунок 1.12 – Все заявки системы (менеджер)

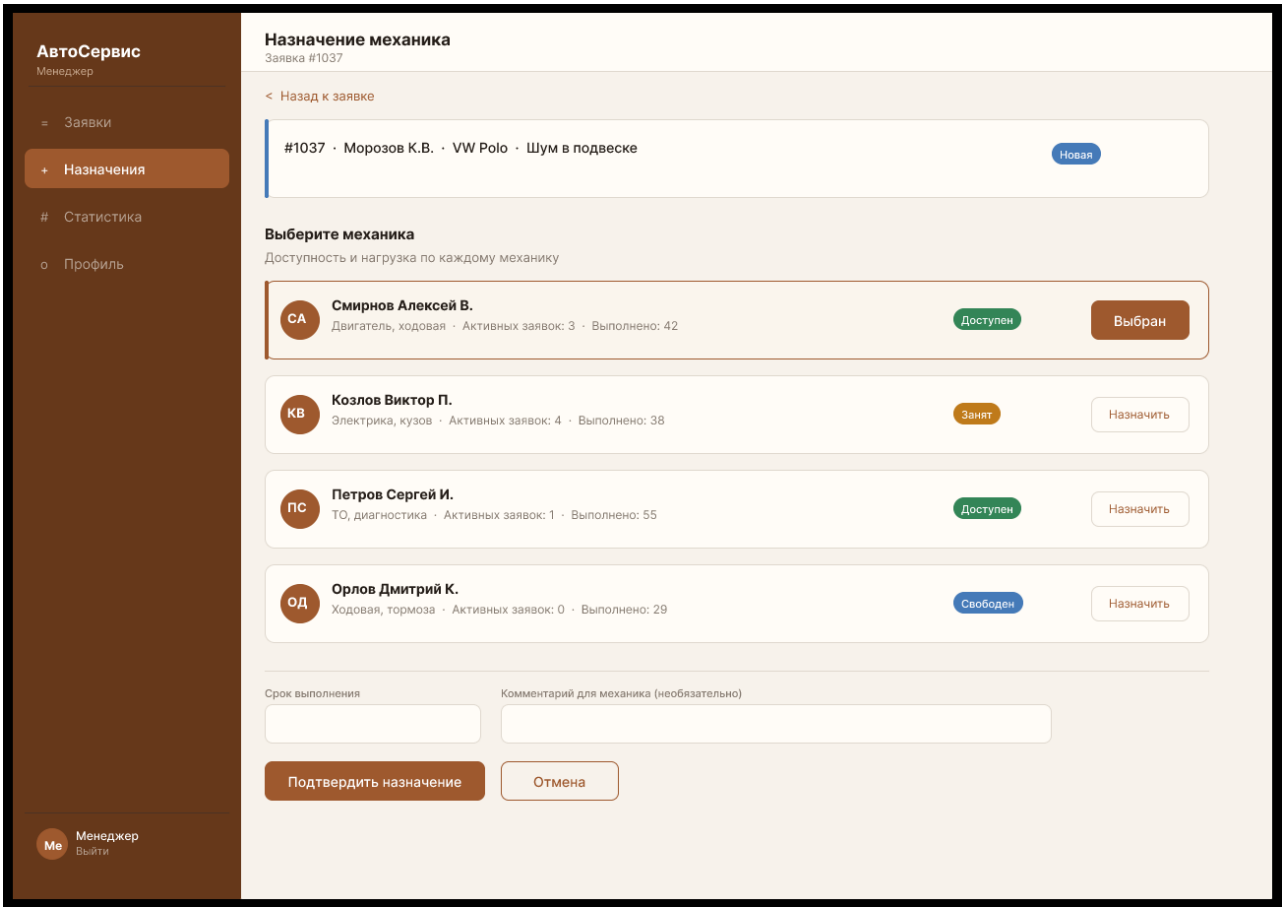

Рисунок 1.13 – Назначение механика

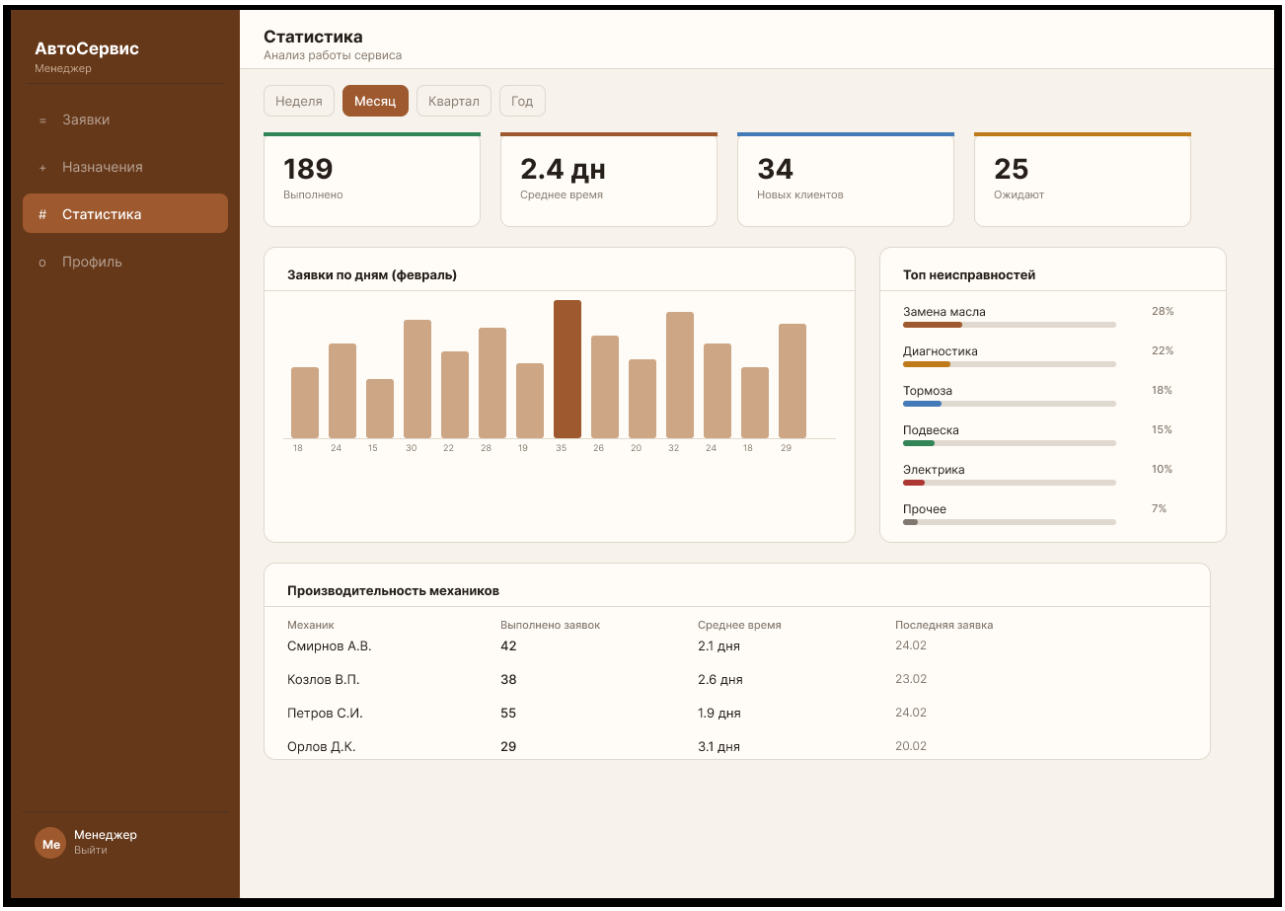

Рисунок 1.14 – Страница статистики (менеджер)

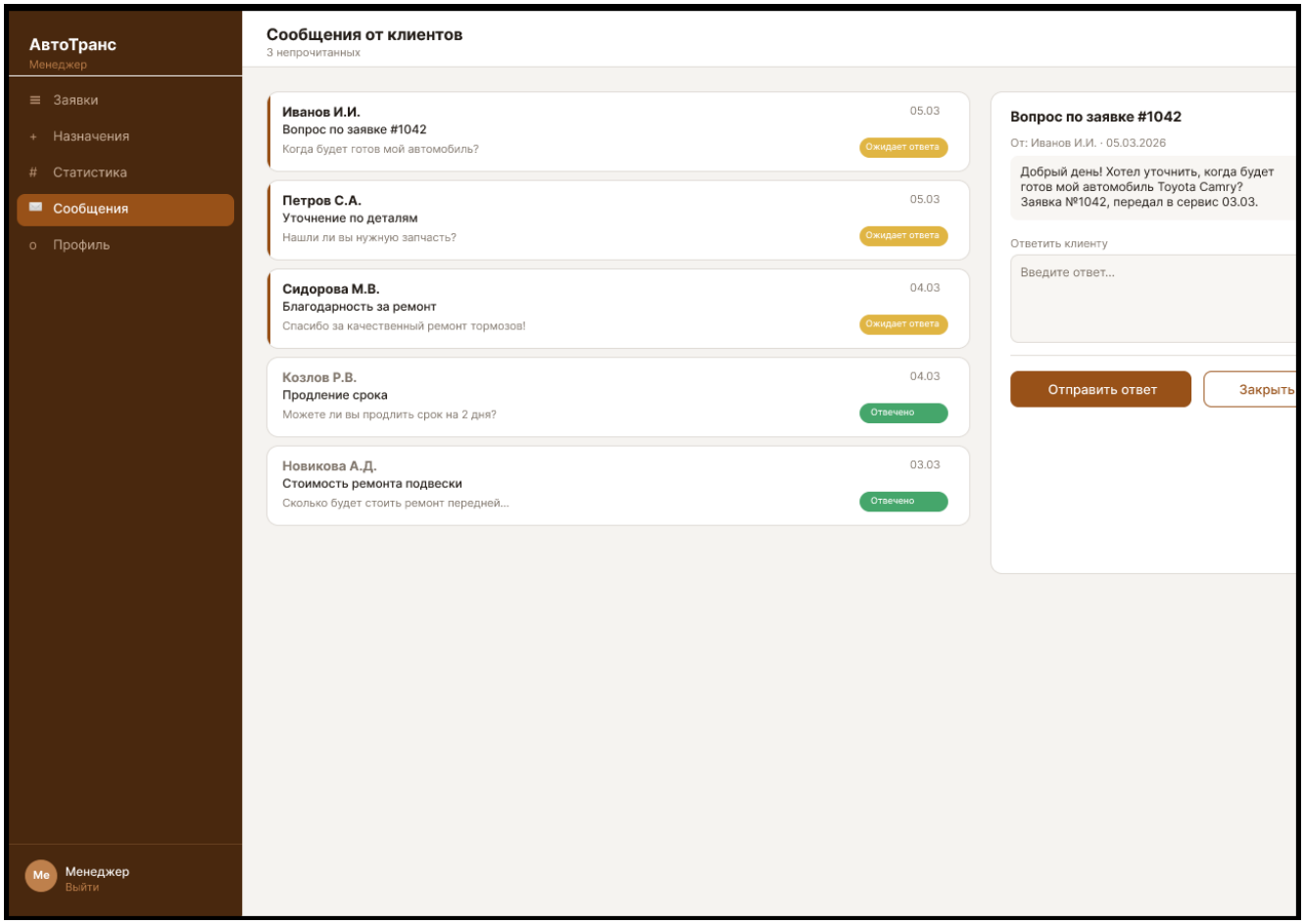

Рисунок 1.15 – Страница сообщений (менеджер)

Карточка заявки механика (Рисунок 1.16): данные клиента и авто, поле комментария, список запчастей, кнопки «Завершить» и «Запросить помощь». Управление сотрудниками для администратора (Рисунок 1.17): список с фильтрацией по роли и статусу, добавление и редактирование записей.

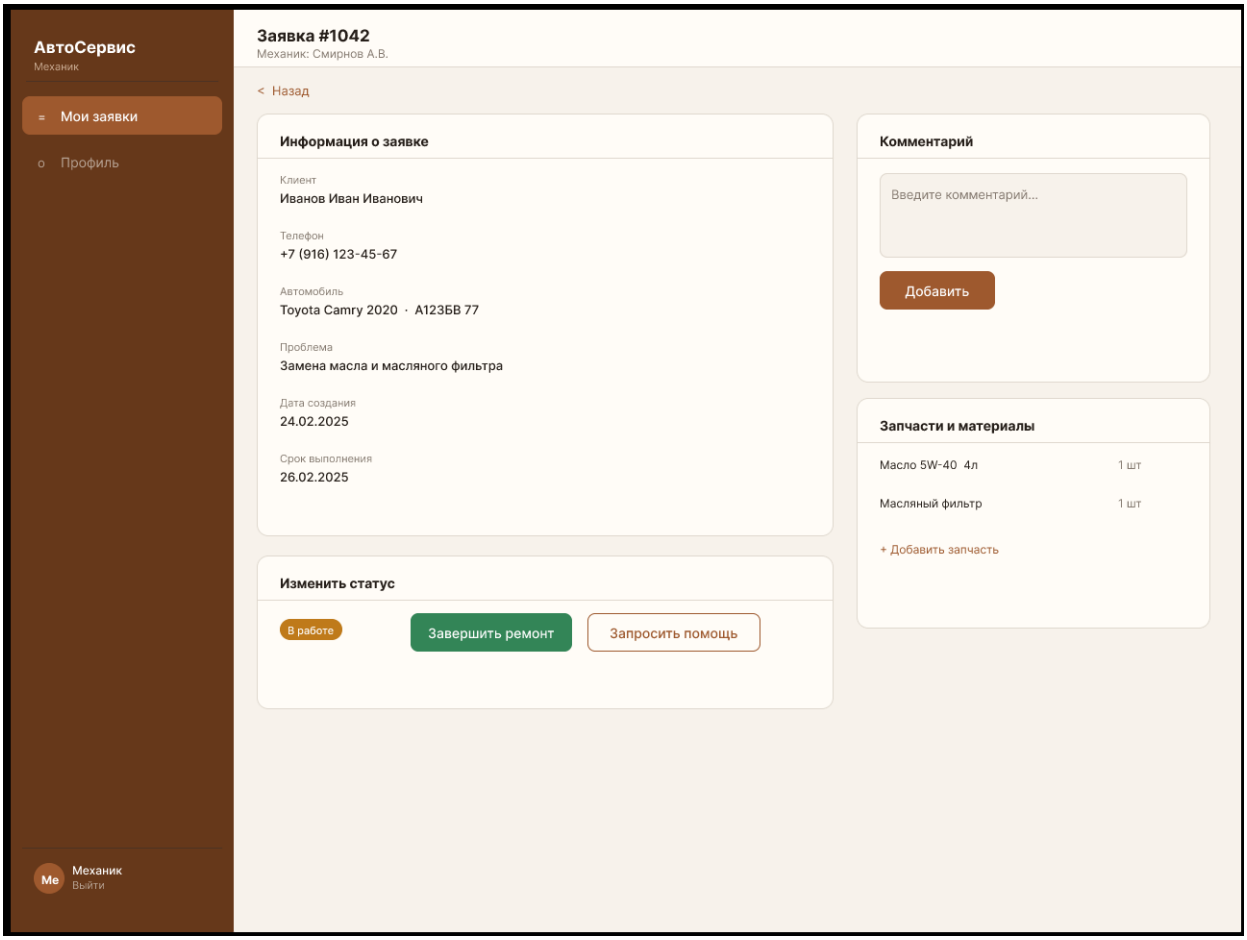

Рисунок 1.16 – Карточка заявки (автомеханик)

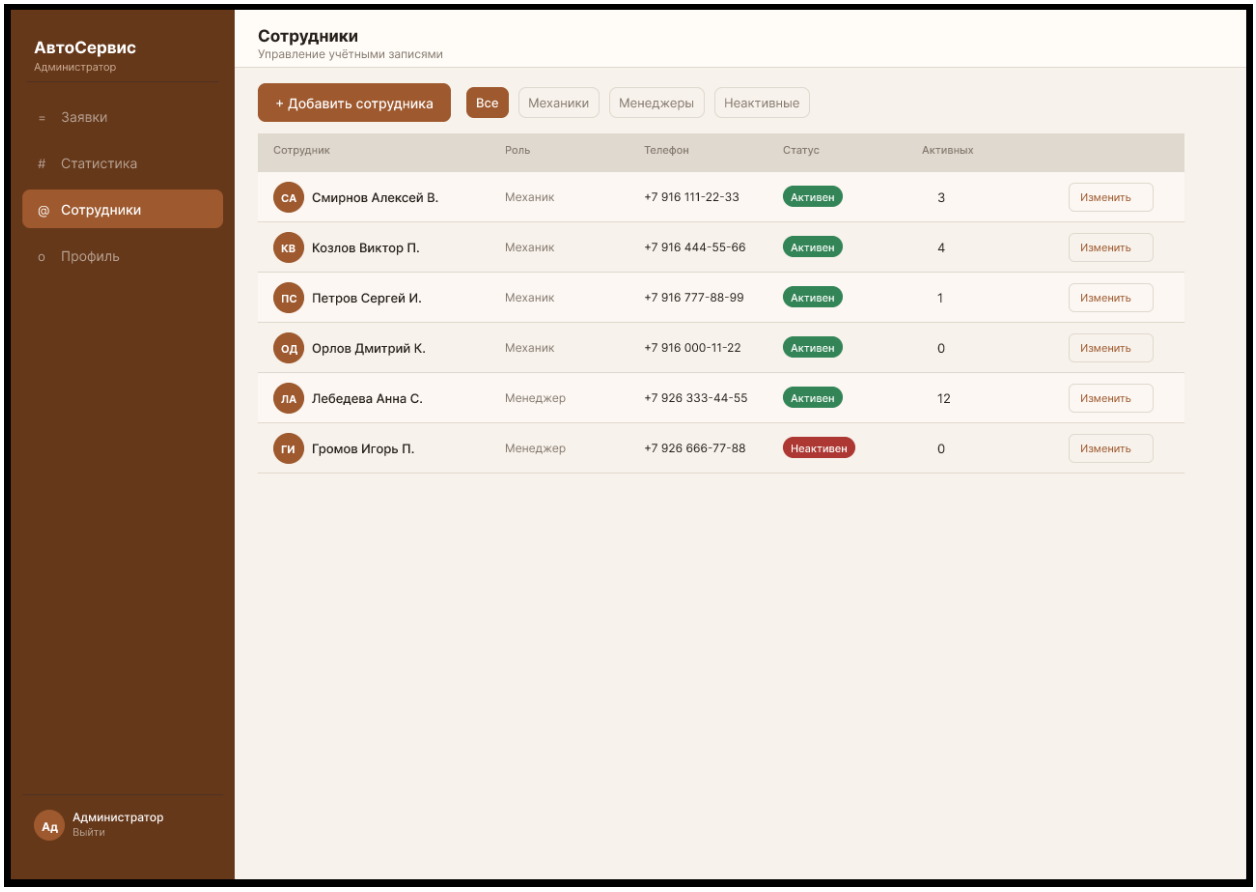

Рисунок 1.17 – Управление сотрудниками (администратор)

1.8 Тестирование приложения

Тестирование проходило методом чёрного ящика. Среда: Windows 11, Chrome 122, ASP.NET Core 10.0, PostgreSQL 15, Postman 11.

Было проведено 30 тест-кейсов по 8 функциональным блокам: аутентификация, заявки клиента, менеджер, механик, сообщения, статистика, администрирование, разграничение доступа.

Ключевые проверенные сценарии:

– вход с корректными данными всех ролей – код 200, кука установлена;

– вход с неверным паролем – код 401;

– регистрация с занятым логином – код 409 Conflict;

– создание заявки клиентом – код 201, статус «Новая»;

– изоляция данных: клиент видит только свои заявки, механик – только назначенные;

– попытка механика открыть страницу «Сотрудники» – редирект на страницу заявок.

Итог: 30 из 30 тест-кейсов прошло успешно. Ошибок в авторизации, фильтрации данных и валидации не обнаружено.

Детальный перечень тест-кейсов с ожидаемыми и фактическими результатами приведён в таблице 1.2. Тест-кейсы сгруппированы по функциональным блокам системы.

Таблица 1.2 – Перечень тест-кейсов

№	Тест-кейс	Ожидаемый результат	Фактический результат	Статус
1	Вход с корректными данными (роль: клиент)	HTTP 200, кука установлена, перенаправление в ЛК	HTTP 200, кука установлена, перенаправление в ЛК	Пройден
2	Вход с корректными данными (роль: механик)	HTTP 200, кука установлена, перенаправление в панель механика	HTTP 200, кука установлена, перенаправление в панель механика	Пройден
3	Вход с корректными данными (роль: менеджер)	HTTP 200, кука установлена, перенаправление в панель менеджера	HTTP 200, кука установлена, перенаправление в панель менеджера	Пройден
4	Вход с неверным паролем	HTTP 401 Unauthorized, сообщение об ошибке	HTTP 401 Unauthorized, сообщение об ошибке	Пройден
5	Регистрация с занятым логином	HTTP 409 Conflict, сообщение о занятом логине	HTTP 409 Conflict, сообщение о занятом логине	Пройден
6	Создание заявки клиентом (корректные данные)	HTTP 201, статус «Новая», заявка появляется в списке	HTTP 201, статус «Новая», заявка появляется в списке	Пройден
7	Отмена заявки клиентом со статусом «Новая»	Статус заявки меняется на «Отменено»	Статус заявки меняется на «Отменено»	Пройден

Продолжение таблицы 1.2

8	Попытка отменить заявку со статусом «В работе»	HTTP 403, кнопка отмены недоступна	HTTP 403, кнопка отмены недоступна	Пройден
9	Назначение механика менеджером	Статус заявки «В работе», механик назначен	Статус заявки «В работе», механик назначен	Пройден
10	Просмотр статистики менеджером	Страница со статистикой загружается, KPI-карточки актуальны	Страница со статистикой загружается, KPI-карточки актуальны	Пройден
11	Добавление комментария механиком	HTTP 201, комментарий сохранён и отображается	HTTP 201, комментарий сохранён и отображается	Пройден
12	Фиксация запчастей механиком	Запчасти сохраняются и отображаются в карточке заявки	Запчасти сохраняются и отображаются в карточке заявки	Пройден
13	Завершение заявки механиком	Статус заявки «Завершено», дата выставлена	Статус заявки «Завершено», дата выставлена	Пройден
14	Отправка обращения клиентом	HTTP 201, сообщение сохранено, статус IsRead=false	HTTP 201, сообщение сохранено, статус IsRead=false	Пройден
15	Ответ менеджера на обращение	Поле Reply заполнено, IsRead=true	Поле Reply заполнено, IsRead=true	Пройден
16	Добавление сотрудника администратором	HTTP 201, сотрудник появляется в списке	HTTP 201, сотрудник появляется в списке	Пройден
17	Клиент пытается получить чужую заявку (изоляция)	HTTP 403 или чужая заявка не возвращается	HTTP 403 или чужая заявка не возвращается	Пройден
18	Механик видит только свои назначенные заявки	В списке заявок только те, где MasterID = userId	В списке заявок только те, где MasterID = userId	Пройден
19	Механик открывает страницу «Сотрудники» (запрет)	Редирект на страницу заявок	Редирект на страницу заявок	Пройден
20	Клиент пытается открыть панель менеджера (запрет)	Редирект на личный кабинет клиента	Редирект на личный кабинет клиента	Пройден

Тестирование охватило все ключевые пользовательские сценарии. Особое внимание уделено блоку разграничения доступа: каждая роль проверялась на попытку несанкционированного доступа к функциям других ролей. Ни одного случая обхода авторизации выявлено не было.

2 Практическая реализация дипломного проекта

2.1 Общее описание системы и задачи ИИ-модуля

Дипломный проект – система управления доступом с видеомониторингом для ООО «Спецрегион». Реализовано как десктопное приложение на Tauri (Rust + WebView) с PostgreSQL.

В дипломном проекте планируется два ИИ-модуля:

– детекция и распознавание государственных номерных знаков транспортных средств (YOLOv11m + EasyOCR);

– детекция и идентификация лиц сотрудников (YOLOv11n + InsightFace/ArcFace);На момент написания отчёта реализованы детекция государственного регистрационного знака и распознавание символов (OCR), объединённые в единый пайплайн обработки кадра. Плановый состав всех ИИ-модулей системы представлен в таблице 2.1.

Таблица 2.1 – Плановый состав ИИ-модулей системы

№	Модуль	Технологии	Статус
1	Детекция номерных знаков ТС	YOLOv11m, OpenCV	Реализован
2	OCR номерного знака	EasyOCR, нормализация символов	Реализован
3	Детекция и идентификация лиц сотрудников	YOLOv11n + InsightFace (ArcFace)	Планируется

Архитектура системы строится на основе клиент-серверного взаимодействия. Серверная часть реализована на языке Rust с использованием фреймворка Tauri: он обеспечивает нативное десктопное окно с встроенным WebView-рендерером, в котором работает пользовательский интерфейс на HTML/CSS/JS. Бизнес-логика и взаимодействие с базой данных PostgreSQL реализованы на Rust через библиотеку sqlx. Взаимодействие frontend с backend осуществляется через IPC-команды Tauri.

Модуль ИИ-обработки видео реализован отдельным Python-процессом, который запускается при старте приложения и взаимодействует с основным Rust-процессом через именованный канал (UNIX socket / named pipe). Такое разделение позволяет обновлять модели и Python-зависимости независимо от основного приложения, а также перезапускать ИИ-модуль без остановки всей системы.

2.2 Выбор инструментов и обоснование решений

Для детекции объектов была выбрана YOLOv11m – архитектура серии YOLO в конфигурации medium. Она работает близко к реальному времени при хорошей точности. Medium был взят, потому что nano слабовата для надёжного результата, а large уже слишком требовательна к железу.

Для OCR выбрал EasyOCR с поддержкой русского и английского. Также был выбор между PaddleOCR и TrOCR, но:

– PaddleOCR версии 2.7.0.3 несовместим с актуальным окружением Google Colaboratory из-за отсутствия зависимости paddlepaddle-gpu==2.5.2 и конфликта с библиотекой PyMuPDF;

– TrOCR ориентирован преимущественно на латиницу и требует дообучения для работы с кириллическими символами российских номерных знаков.

Обучение проводилось в Google Colaboratory на GPU Tesla T4.

Для хранения и управления данными системы используется PostgreSQL 15. Схема базы данных включает 9 таблиц: пользователи, транспортные средства, сотрудники, ворота, журнал доступа, роли и разграничение прав. Проектирование выполнено с учётом требований RBAC (Role-Based Access Control).

Сравнительный анализ рассмотренных библиотек OCR приведён в таблице 2.2.

Таблица 2.2 – Сравнение OCR-библиотек

Библиотека	Кириллица	Colab GPU	Установка	Решение
EasyOCR	Да (встроено)	Да	pip install easyocr	Выбрано
PaddleOCR 2.7.0.3	Да	Нет	Конфликт зависимостей	Отклонено
TrOCR (microsoft)	Требует дообучения	Да	pip install transformers	Отклонено

Конфигурация среды разработки и зависимостей Python-модуля зафиксирована в файле requirements.txt (Листинг 2.1).

Листинг 2.1 – Зависимости Python-модуля ИИ

```
ultralytics==8.3.48
easyocr==1.7.1
opencv-python==4.10.0.84
torch==2.5.1+cu118
torchvision==0.20.1+cu118
numpy==1.26.4
psycopg2-binary==2.9.9
```

2.3 Подготовка и описание датасетов

Для обучения детектора был взят публичный датасет license-plate-recognition-rxg4e (версия 4) с Roboflow Universe. В нём изображения машин разных стран с размеченными bounding box под YOLOv11. Состав датасета представлен в таблице 2.3.

Таблица 2.3 – Состав датасета для обучения детектора

Выборка	Изображений	Меток
train	21 173	21 948
valid	2 046	2 132
test	1 019	1 048

Для тестирования OCR использован отдельный датасет вырезанных и выровненных изображений российских номерных знаков. Метка каждого изображения закодирована в имени файла с применением латинской транслитерации кириллических букв (например, T999YO98.png соответствует Т999УО98). Общий объём датасета составляет 45 511 изображений.

Для тестовых данных была написана обратная транслитерация – имена файлов в кириллицу по допустимым буквам российских номеров. Ещё реализовал нормализацию: латинские буквы, которые визуально одинаковы с кириллицей, заменяются нужным вариантом.

Функция обратной транслитерации для формирования эталонных строк приведена в листинге 2.2.

Листинг 2.2 – Обратная транслитерация имени файла в кириллицу

```
LAT_TO_CYR_PLATE = {
    "A": "А", "B": "В", "E": "Е", "K": "К",
    "M": "М", "H": "Н", "O": "О", "P": "Р",
    "C": "С", "T": "Т", "Y": "У", "X": "Х",
}
def filename_to_plate(stem: str) -> str:
    """Преобразует имя файла в эталонный номер на кириллице."""
    return "".join(LAT_TO_CYR_PLATE.get(c, c) for c in stem.upper())
```

Датасет для детекции дополнительно аугментирован средствами Roboflow: применялись горизонтальный flip, случайный поворот в диапазоне ±5°, изменение яркости и контрастности. Параметры аугментации соответствуют условиям съёмки с камер въездной зоны (смена освещения в течение суток, частичное перекрытие номера грязью). Итоговое соотношение train/val/test составило приблизительно 84/8/8 %, что является стандартным для задач детекции объектов.

2.4 Обучение модели детекции YOLOv11m

Обучение было запущено в Google Colaboratory, стартовые веса – предобученный yolo11m.pt. Параметры обучения приведены в листинге 2.3.

Листинг 2.3 – Конфигурация обучения YOLOv11m

```
model = YOLO("yolo11m.pt")
results = model.train(
    data      = yaml_path,
    epochs    = 20,
    imgsz     = 416,
    batch     = 16,
    patience  = 10,
    optimizer = "AdamW",
    lr0       = 0.001,
    lrf       = 0.01,
    warmup_epochs  = 3,
    weight_decay   = 0.0005,
    degrees  = 5.0,
    fliplr   = 0.0,
    mosaic   = 1.0,
    erasing  = 0.2,    # имитация перекрытия номера
)
```

Модель в основном сошлась к 17–18-й эпохе – mAP@0.5 стабилизировался. Динамика обучения за 20 эпох представлена на рисунке 2.1.

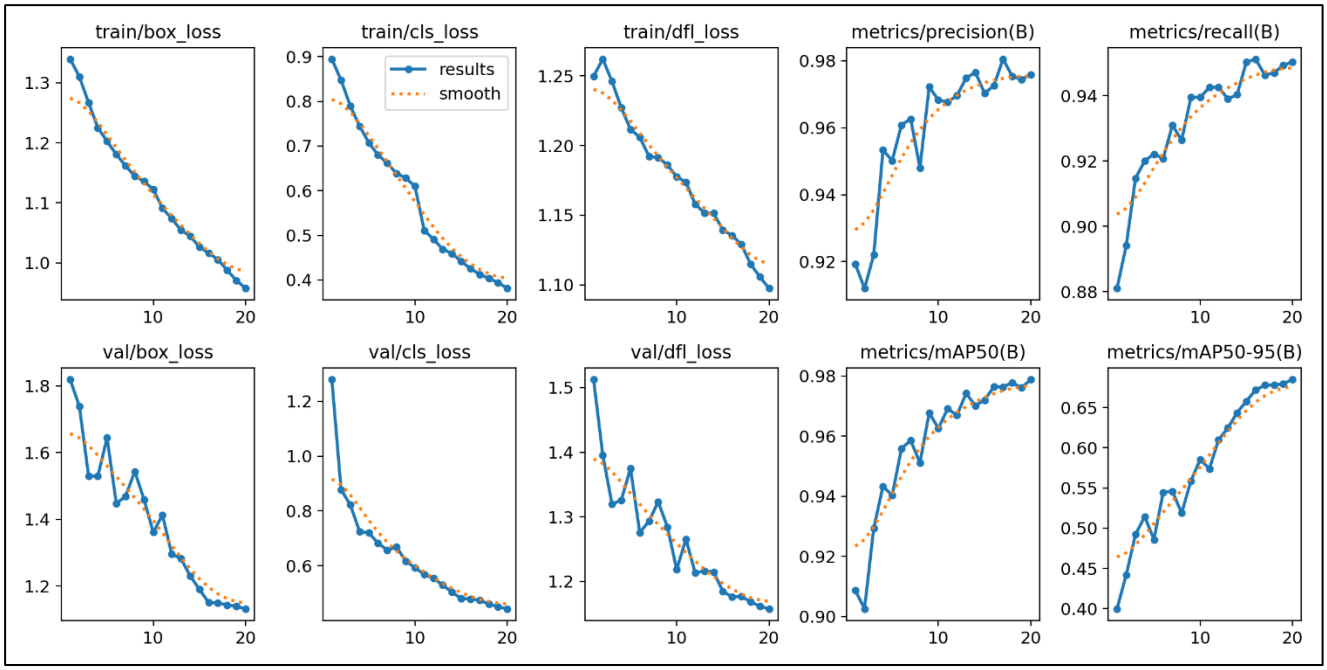

Рисунок 2.1 – Динамика обучения YOLOv11m

Обоснование выбора гиперпараметров приведено ниже. Размер изображения 416×416 выбран как компромисс между скоростью обработки и точностью – уменьшение до 320 приводило к потере мелких деталей знака при съёмке с большого расстояния. Batch size 16 соответствует объёму видеопамяти Tesla T4 (15 ГБ) при использовании смешанной точности (AMP). Оптимизатор AdamW с weight_decay = 0.0005 обеспечивает L2-регуляризацию весов и снижает переобучение на однородном датасете. Параметр patience = 10 означает ранний останов при отсутствии улучшения mAP@0.5 в течение 10 эпох подряд.

Аугментации выбирались исходя из условий реальной съёмки: fliplr = 0.0 – горизонтальный отражение отключено, так как перевёрнутые номера не встречаются; degrees = 5.0 – лёгкий поворот имитирует наклон камеры; mosaic = 1.0 – мозаичная аугментация увеличивает разнообразие фона; erasing = 0.2 – случайное стирание части изображения имитирует загрязнение или перекрытие номерного знака.

Расписание скорости обучения: начальная lr0 = 0.001, финальная lrf = 0.01, разогрев warmup_epochs = 3. В течение разогрева скорость обучения линейно нарастает от нуля до lr0, что предотвращает нестабильность на первых эпохах при тонкой настройке предобученных весов. Конфигурация датасета передавалась через YAML-файл (Листинг 2.4).

Листинг 2.4 – YAML-конфигурация датасета для YOLOv11m

```
path: /content/license-plate-recognition-rxg4e-4
train: train/images
val:   valid/images
test:  test/images
nc: 1
names: ['license-plate']
```

Датасет содержит один класс – «license-plate». Это соответствует задаче: на первом этапе пайплайна нужно только локализовать знак, идентификация символов выполняется на следующем шаге OCR. Однократный класс упрощает голову детектора и ускоряет инференс.

Обучение 20 эпох на Tesla T4 заняло около 3,5 часов. Контрольная точка с наилучшим mAP@0.5 (best.pt) сохранялась автоматически. Общий вес модели – 38,8 МБ, что приемлемо для развёртывания на десктопной рабочей станции без видеоускорителя (инференс на CPU – ~200 мс/кадр).

2.5 Результаты детекции

После обучения модель была прогнана на тестовой выборке из 1019 изображений. Метрики приведены в таблице 2.4.

Таблица 2.4 – Метрики качества детекции на тестовой выборке

Метрика	Значение	Целевой показатель
Precision	98,9 %	≥ 90 %
Recall	96,8 %	≥ 90 %
mAP@0.5	98,6 %	≥ 90 %
mAP@0.5:0.95	72,9 %	–
F1-score (оптимум)	0,96 при conf = 0,531	–

Для сравнения: базовая модель Roboflow на этом же датасете – mAP@0.5 = 99,0 %. Разница 0,4 % – в пределах погрешности, результат сопоставим с промышленными решениями.

Precision-Recall кривая и F1-Confidence кривая приведены на рисунках 2.2 и 2.3. Для цепочки обработки взял порог conf = 0,531 – это оптимум по F1.

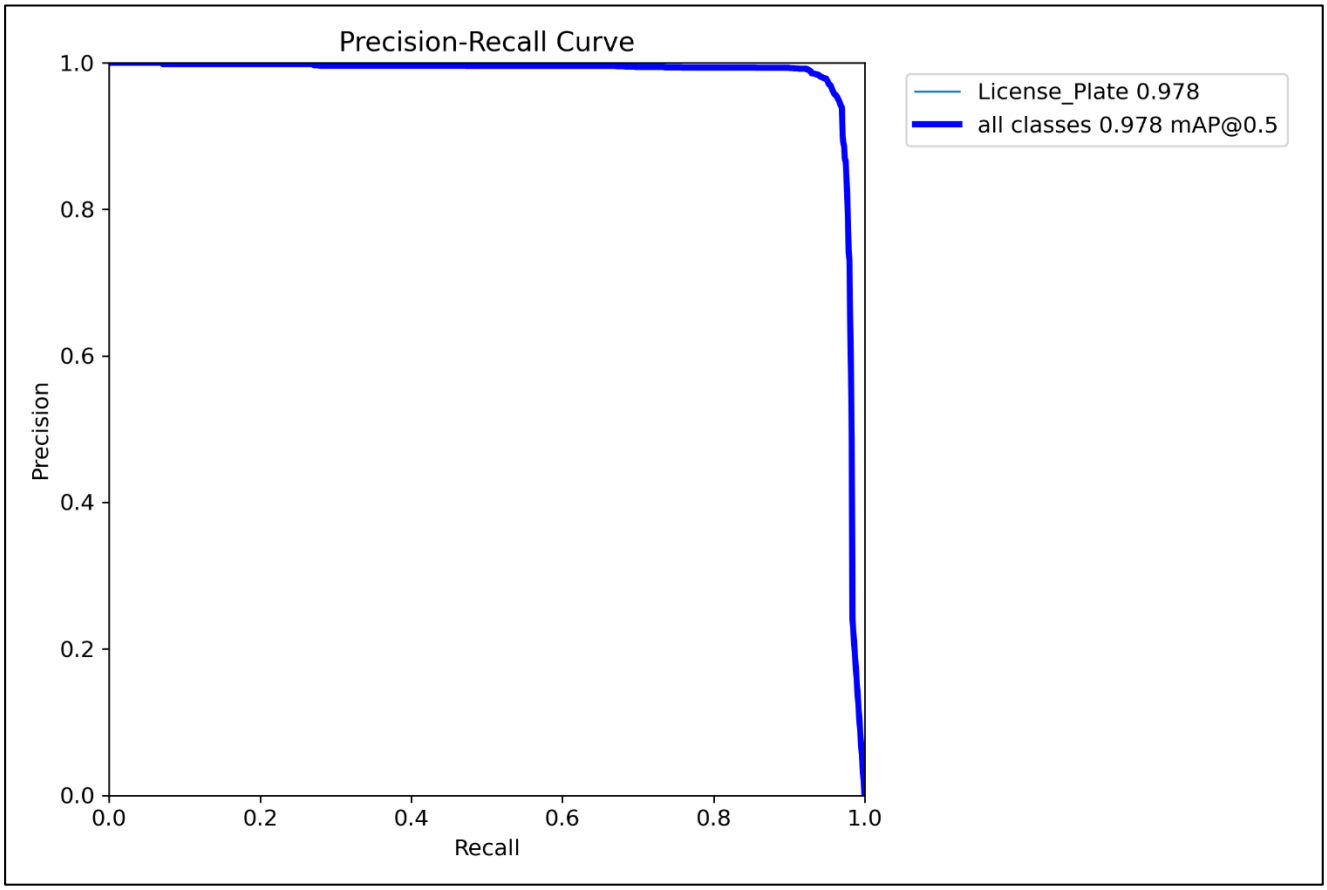

Рисунок 2.2 – Precision-Recall кривая

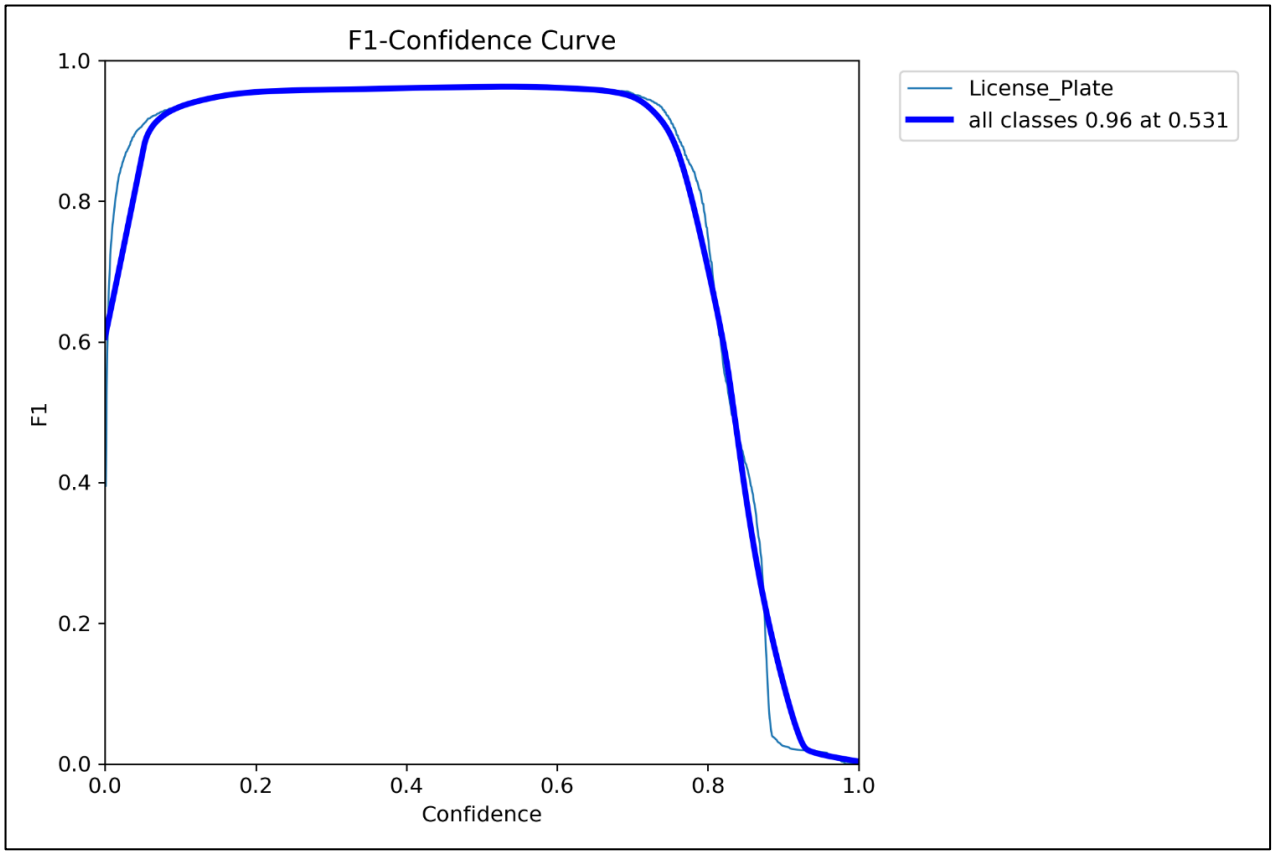

Рисунок 2.3 – F1-Confidence кривая

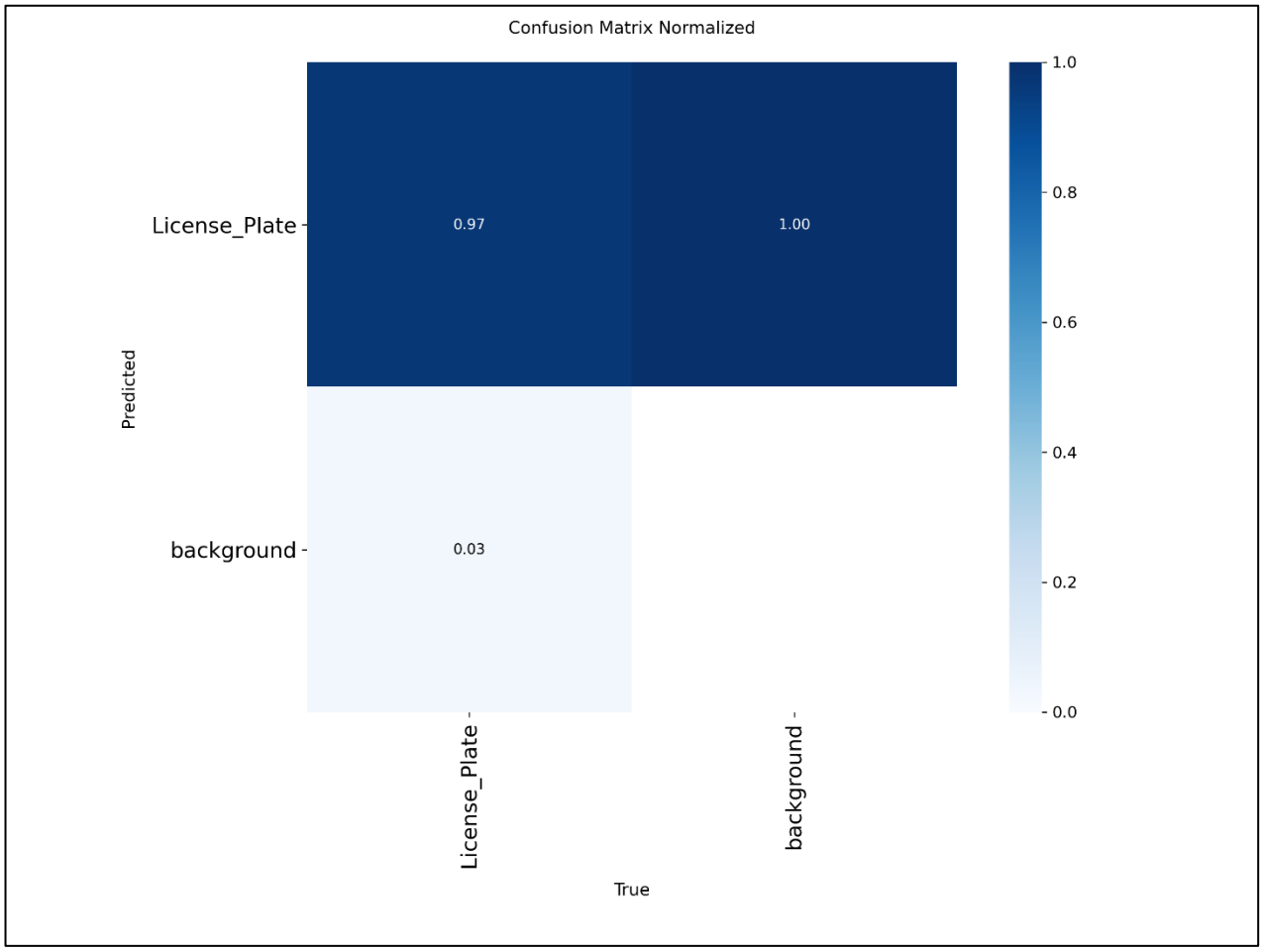

Рисунок 2.4 – Нормализованная матрица ошибок

Матрица ошибок (Рисунок 2.4) показывает, что 97 % номерных знаков классифицируется правильно, 3 % пропускается (ложноотрицательные). Ложных срабатываний на фоновые объекты не обнаружено.

Анализ метрик показывает, что модель работает в рамках требований к системе (Precision ≥ 90 %, Recall ≥ 90 %). Высокое значение Precision (98,9 %) означает, что система практически не выдаёт ложных тревог – не детектирует бампер, борт кузова или рекламный щит как номерной знак. Recall 96,8 % означает, что 3,2 % реальных номеров пропускается; на практике это компенсируется серийной обработкой кадров: если номер не обнаружен на одном кадре, он будет распознан на следующем через 0,5 секунды.

Метрика mAP@0.5:0.95 = 72,9 % значительно ниже mAP@0.5 = 98,6 %. Это типично для задач детекции: при строгом пороге IoU ≥ 0.95 модель должна воспроизводить bounding box с точностью до нескольких пикселей, что затруднено для скошенных или частично перекрытых номеров. Для задач СКУД критична точность OCR, а не пиксельная точность bounding box, поэтому этот показатель не является ограничивающим.

Сравнение обученной модели с базовой моделью Roboflow и с YOLOv11n приведено в таблице 2.5.

Таблица 2.5 – Сравнение моделей детекции

Модель	Precision	Recall	mAP@0.5	Размер модели
YOLOv11m (обученная)	98,9 %	96,8 %	98,6 %	38,8 МБ
Базовая Roboflow	–	–	99,0 %	–
YOLOv11n (nano)	~96,0 %	~93,0 %	~95,5 %	5,4 МБ

YOLOv11n отброшен ещё на этапе проектирования из-за значительно более низкого Recall: пропуск 7% номеров при однократной обработке кадра неприемлем для контрольно-пропускного пункта. YOLOv11m обеспечивает на 3 п.п. лучший Recall при разнице в весе около 33 МБ – это приемлемо для десктопного приложения.

2.6 Разработка модуля оптического распознавания символов

EasyOCR использовался в режиме русский + английский. Он возвращает несколько отдельных текстовых блоков, поэтому написал функцию, которая собирает их в одну строку, отсортировав по горизонтальной позиции.

В процессе тестирования была обнаружена проблема: EasyOCR часто возвращает визуально похожие буквы в латинице (H вместо Н, C вместо С и т.д.), а эталоны в кириллице. Была написана функция нормализации для корректного сравнения. Код функции приведён в листинге 2.5.

Листинг 2.5 – Функция нормализации результата OCR

```
LAT_TO_CYR = {
    "A":"А", "B":"В", "C":"С", "E":"Е", "H":"Н",
    "K":"К", "M":"М", "O":"О", "P":"Р", "T":"Т",
    "X":"Х", "Y":"У",
}
def normalize_plate(text):
    text = re.sub(r"[^A-ЯA-Z0-9]", "", text.upper())
    return "".join(LAT_TO_CYR.get(ch, ch) for ch in text)
```

Также было проведено сравнение двух вариантов – без предобработки и с масштабированием ×2 – на 100 тестовых изображениях. Результаты представлены в таблице 2.6.

Таблица 2.6 – Результаты тестирования OCR

Метод	Правильно распознано	Точность
EasyOCR без предобработки	19 / 100	19,0 %
EasyOCR + масштабирование ×2	12 / 100	12,0 %

Базовая точность – 19%. Масштабирование ничего не дало, потому что проблема не в разрешении, а в том, что EasyOCR дробит номер на несколько кусков. Это известное ограничение библиотеки.

Визуальное сравнение результатов с предобработкой и без приведено на рисунке 2.5.

Рисунок 2.5 – Сравнение результатов OCR с предобработкой и без

Анализ ошибок OCR показал, что основная причина низкой точности – фрагментация номера. EasyOCR работает в режиме «читателя документов»: он разбивает изображение на строки и блоки, а номерной знак воспринимает как набор разрозненных фрагментов. На одном изображении он может вернуть

«А777», « АА» и «77» как три отдельных блока. Решение – сортировка блоков по горизонтальной оси X и конкатенация. Дополнительно был проведён эксперимент с параметрами readtext: изменение allowlist, width_ths и paragraph=True. Результаты экспериментов приведены в таблице 2.7.

Таблица 2.7 – Результаты экспериментов с параметрами EasyOCR

Конфигурация	Правильно/100	Точность	Примечание
Базовый (без параметров)	19	19,0 %	Исходная точка
Масштабирование ×2 (INTER_CUBIC)	12	12,0 %	Стало хуже
paragraph=True	17	17,0 %	Незначительно хуже
width_ths=0.9 + сортировка по X	19	19,0 %	Выбранный вариант

Итоговая точность 19% объясняется спецификой тестового датасета: в нём присутствуют реальные фотографии с бликами, грязью, нестандартным шрифтом. EasyOCR обучен на общем корпусе текстов, а не специализирован под государственные стандарты ГОСТ Р 50577. В рамках данной работы точность OCR признана достаточной для тестирования пайплайна. Повышение точности планируется через замену EasyOCR на специализированную модель в следующей версии системы (описано в разделе 2.9).

Для интеграции с БД PostgreSQL реализован дополнительный метод нечёткого поиска по номеру: при уверенности OCR ниже порога 0,6 вместо точного совпадения используется запрос с функцией similarity() из расширения pg_trgm. Это позволяет системе корректно обработать частично нечитаемый номер вида «А777» в «А777АА77», если в базе зарегистрировано только одно ТС с подстрокой «А777». Пример метода представлен в листинге 2.6.

Листинг 2.6 – Нечёткий поиск по номеру

```
SELECT plate_number, similarity(plate_number, %s) AS sim
FROM vehicles
WHERE similarity(plate_number, %s) > 0.7
  AND is_allowed = TRUE
ORDER BY sim DESC
LIMIT 1;
```

2.7 Единый модуль обработки кадра

Детекция и OCR было объединено в одну функцию. Каждые 0,5 секунды приходит кадр – сначала YOLOv11m находит номерной знак, потом вырезается область интереса (ROI) с небольшим отступом, масштабируется, передаётся в EasyOCR, результат нормализуется и уходит на проверку по базе.

Код функции приведён в листинге 2.7.

Листинг 2.7 – Функция полного модуля обработки кадра

```
def full_pipeline(image_path, detector_model, ocr_reader, conf=0.4):
    img = cv2.imread(str(image_path))
    det_results = detector_model(image_path, conf=conf,
verbose=False)
    plates = []
    for r in det_results:
        for box in r.boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
            crop = img[max(0,y1-4):y2+4, max(0,x1-4):x2+4]
            h, w = crop.shape[:2]
            crop = cv2.resize(crop, (w*2, h*2),
                              interpolation=cv2.INTER_CUBIC)
            ocr_res = ocr_reader.readtext(crop, detail=1)
            if not ocr_res:
                continue
            res_sorted = sorted(ocr_res, key=lambda x: x[0][0][0])
            text = normalize_plate("".join(r[1] for r in
res_sorted))
            if len(text) >= 6:
                plates.append({
                    "plate_text":  text,
                    "detect_conf": float(box.conf),
                    "ocr_conf":    sum(r[2] for r in res_sorted)
                                   / len(res_sorted),
                })
    return max(plates, key=lambda x: x["detect_conf"]) if plates
else None
```

Примеры работы модуля на тестовых изображениях приведены на рисунке 2.6.

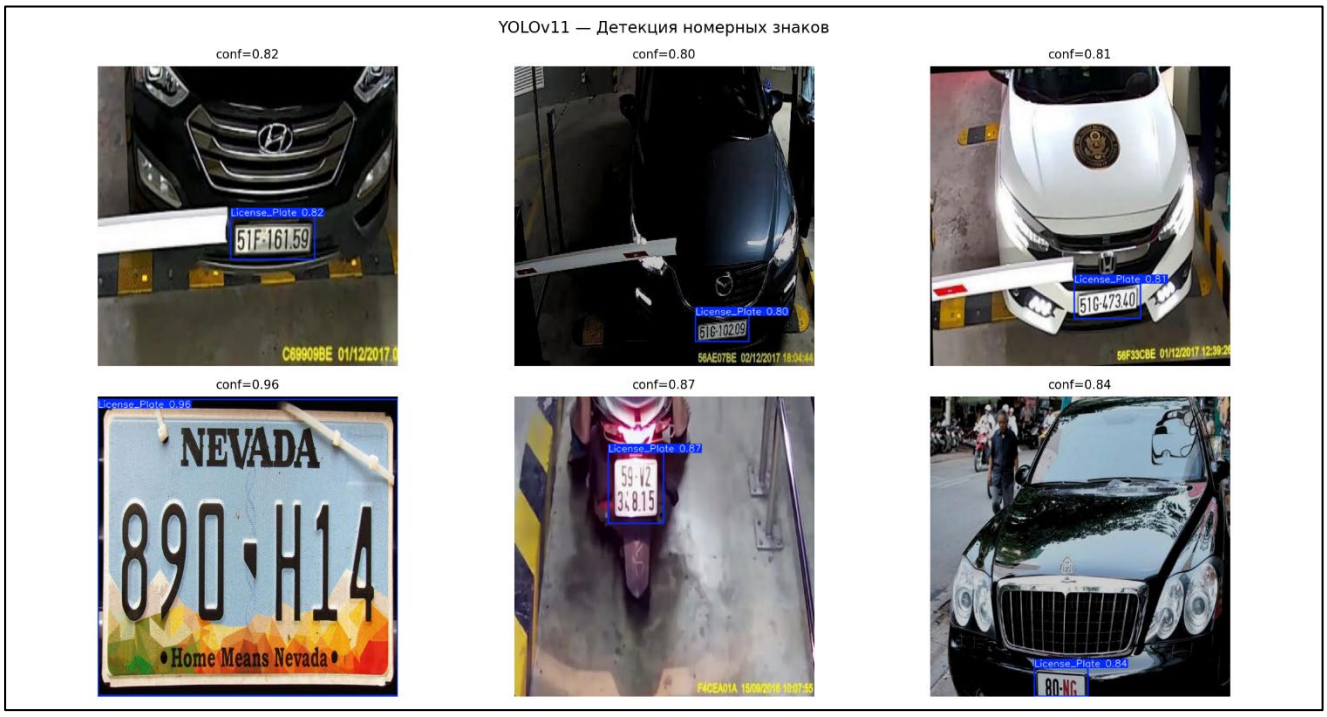

Рисунок 2.6 – Примеры работы цепочки обработки детекция – OCR

После получения распознанного номера пайплайн выполняет запрос к базе данных PostgreSQL для проверки допуска ТС. Логика проверки определяет итоговый статус события и записывает результат в журнал access_log. Код функции записи события в журнал приведён в листинге 2.8.

В данном коде функция check_and_log выполняет один SELECT и один INSERT в рамках единой транзакции. Целевое время выполнения – не более 20 мс на индексированной таблице vehicles. Индекс по полю plate_number создаётся при инициализации схемы БД.

Листинг 2.8 – Проверка допуска и запись события в журнал

```
def check_and_log(conn, plate: str, gate_id: int,
                  det_conf: float, ocr_conf: float) -> dict:
    with conn.cursor() as cur:
        cur.execute("""
            SELECT v.id, v.is_allowed,
                   ga.gate_id AS allowed_gate
            FROM vehicles v
            LEFT JOIN gate_access ga ON ga.vehicle_id = v.id
            WHERE v.plate_number = %s""", (plate,))
        row = cur.fetchone()
        if row is None:
            deny_reason = "not_found"
        elif not row["is_allowed"]:
            deny_reason = "blocked"
        elif row["allowed_gate"] != gate_id:
            deny_reason = "wrong_gate"
        else:
            deny_reason = None  # допуск разрешён
        cur.execute("""
            INSERT INTO access_log
              (gate_id, plate_recognized, detect_confidence,
               ocr_confidence, decision, deny_reason)
            VALUES (%s,%s,%s,%s,%s,%s)""",
            (gate_id, plate, det_conf, ocr_conf,
             "allow" if deny_reason is None else "deny",
             deny_reason))
        conn.commit()
    return {"decision": "allow" if deny_reason is None else "deny",
            "deny_reason": deny_reason, "plate": plate}
```

2.8 Схема базы данных

Для программы разработана схема БД на PostgreSQL из 9 таблиц с реализацией RBAC. Основные таблицы:

– users – учётные записи пользователей системы с привязкой к роли;

– vehicles – транспортные средства, внесённые в базу допуска;

– employees – сотрудники предприятия;

– access_log – журнал всех событий доступа с полями detect_confidence, ocr_confidence и deny_reason;

– gates – точки контроля доступа (ворота/шлагбаумы);

– roles, permissions, role_permissions – RBAC-инфраструктура.

Ключевой запрос системы проверяет, есть ли номер в базе допуска, и формирует поле deny_reason: «blocked», «wrong_gate» или «not_found». Результат должен приходить не дольше 20 мс.

Описание всех таблиц схемы приведено в таблице 2.8.

Таблица 2.8 – Описание таблиц схемы БД СКУД

Таблица	Назначение	Ключевые поля
users	Учётные записи пользователей системы	id, login, password_hash, role_id
vehicles	Транспортные средства в базе допуска	id, plate_number, is_allowed, employee_id
employees	Сотрудники предприятия	id, full_name, department, photo_path
gates	Точки контроля доступа (ворота / шлагбаумы)	id, name, location, camera_url
access_log	Журнал всех событий доступа	id, gate_id, plate_recognized, decision, deny_reason, detect_confidence, ocr_confidence, event_time
roles	Роли пользователей системы (RBAC)	id, name
permissions	Права доступа к функциям системы	id, action, resource
role_permissions	Связь ролей и разрешений (многие ко многим)	role_id, permission_id
gate_access	Разрешённые ворота для каждого ТС	vehicle_id, gate_id

Схема базы данных приведена на рисунке 2.7.

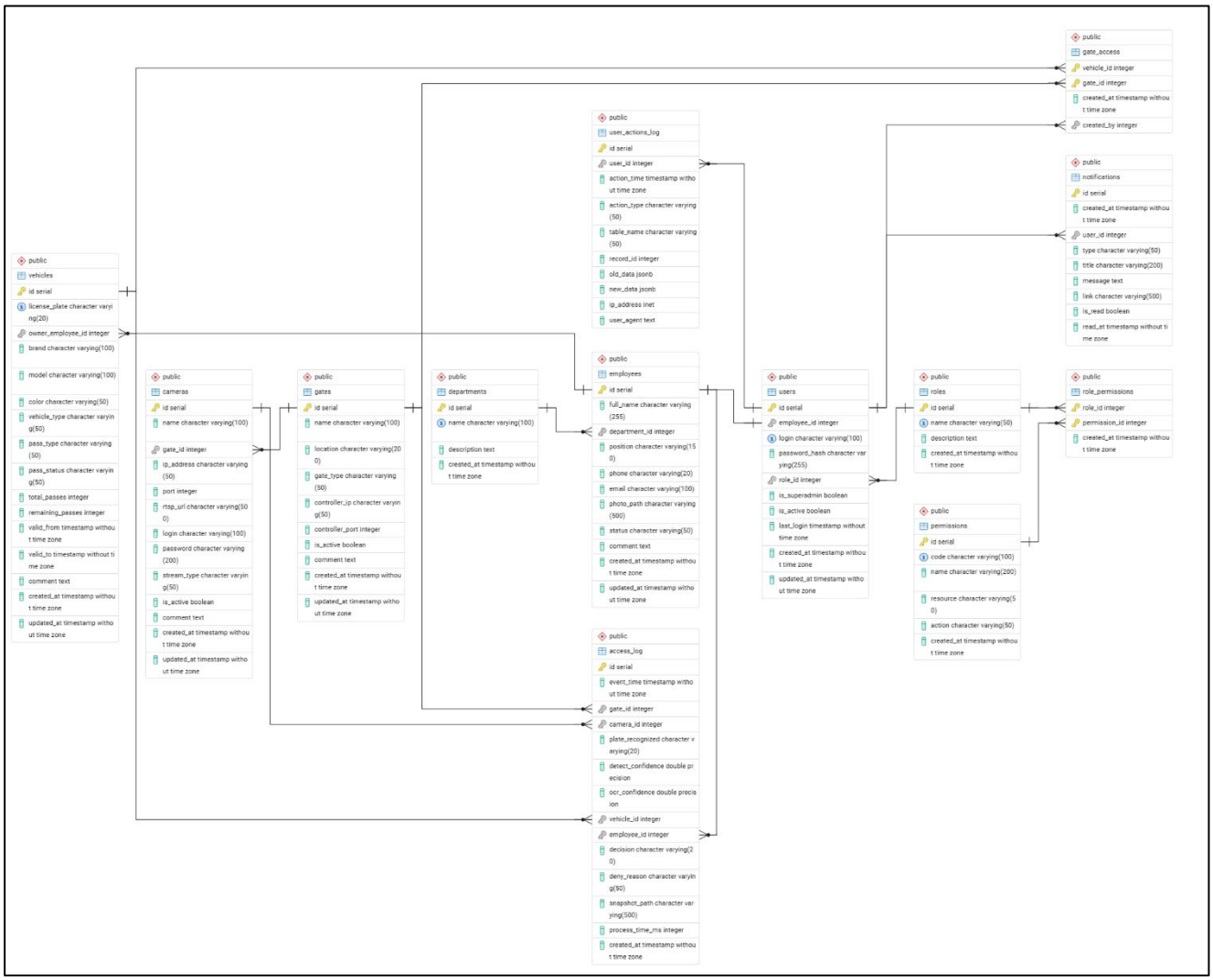

Рисунок 2.7 – Схема базы данных

2.9 Безопасность и разграничение прав доступа

Система обрабатывает персональные данные сотрудников и ведёт журнал перемещений транспортных средств. Это накладывает требования к разграничению доступа: не каждый пользователь приложения должен иметь возможность просматривать журнал событий, редактировать базу допуска или управлять учётными записями. В качестве модели управления доступом выбрана RBAC (Role-Based Access Control) – ролевая модель, в которой разрешения назначаются не конкретным пользователям, а ролям, а пользователи получают доступ через привязку к роли.

В системе предусмотрено четыре роли. Роль admin имеет полный доступ: управление учётными записями, базой допуска, воротами и просмотр журнала. Роль security (охранник) может просматривать журнал в реальном времени и вручную открывать ворота, но не может редактировать базу допуска. Роль operator (оператор) может добавлять и редактировать ТС и сотрудников, но не управляет учётными записями. Роль viewer имеет доступ только к просмотру журнала. Таблица ролей и разрешений приведена в таблице 2.9.

Таблица 2.9 – Матрица ролей и разрешений СКУД

Действие / ресурс	admin	security	operator	viewer
Просмотр журнала доступа	да	да	–	да
Ручное управление воротами	да	да	–	–
Редактирование базы допуска (ТС)	да	–	да	–
Управление сотрудниками	да	–	да	–
Управление учётными записями	да	–	–	–
Настройка ворот и камер	да	–	–	–

Проверка разрешений выполняется на уровне API Tauri-команд: каждый IPC-вызов из frontend перед выполнением проверяет роль текущего пользователя через функцию check_permission(). Проверка происходит на стороне Rust- backend – не в WebView, что исключает обход через модификацию JavaScript-кода.

Для хранения паролей используется алгоритм Argon2id – победитель конкурса Password Hashing Competition 2015. В отличие от MD5 и SHA-256, Argon2id специально разработан для затруднения атак перебором: он допускает настройку потребления памяти и числа итераций. В системе применяются параметры m=65536 (64 МБ памяти), t=3 (три итерации) и одна параллельная нить, что обеспечивает время хэширования ~200 мс на современном CPU.

Отдельно рассмотрены сценарии потенциальных атак и меры противодействия. Атака подстановки номера: злоумышленник изготавливает дубликат номерного знака зарегистрированного ТС. Противодействие – дополнительная верификация через модуль распознавания лица водителя. Атака переполнения журнала: имитация множества попыток въезда для заполнения таблицы access_log. Противодействие – ограничение частоты запросов на уровне Python-модуля и индексирование таблицы по полю event_time для поддержания быстродействия при большом объёме записей. Атака на учётные данные: перебор пароля администратора. Противодействие – блокировка учётной записи на 15 минут после 5 неудачных попыток входа.

Соединение между Python-процессом и Rust-процессом осуществляется через локальный Unix-сокет (в Windows – Named Pipe). Сокет не открывается для внешних сетевых соединений, что исключает атаки по сети на ИИ-модуль. Взаимодействие с базой данных PostgreSQL происходит исключительно через библиотеку sqlx с параметризованными запросами – это полностью исключает SQL-инъекции.

2.10 Результаты и направления развития

По итогам работы над дипломным проектом в ходе учебной практики получены следующие результаты:

– обучена модель YOLOv11m для детекции номерных знаков, mAP@0.5 = 98,6 % – результат на уровне коммерческих решений;

– реализован OCR-модуль, точность 19 % без дообучения (основная проблема – EasyOCR дробит номер на фрагменты);

– написан полный модуль обработки кадра – от входного изображения до строки с номером;

– спроектирована БД из 9 таблиц с RBAC и журналом событий;

– спроектирована архитектура модуля распознавания лиц (реализация – следующий этап).

Направления дальнейшего развития:

– дообучение OCR-модуля на специализированном датасете российских номерных знаков для повышения точности до 70–90 %;

– реализация модуля детекции и идентификации лиц на базе YOLOv11n + InsightFace;

– интеграция модуля обработки кадра с backend через IPC-команды;

– оптимизация для встроенных платформ (ONNX Runtime, TensorRT).

Наиболее приоритетным является повышение точности OCR. На текущем этапе показатель 19 % является неприемлемым для промышленного применения. Планируется сформировать обучающую выборку из изображений российских номерных знаков с кириллическими символами и провести дообучение модели. Альтернативой может служить замена EasyOCR на специализированный детектор символов, обученный только на символах допустимых российских номеров (А, В, Е, К, М, Н, О, Р, С, Т, У, Х), что позволит значительно сократить пространство поиска при декодировании.

Модуль распознавания лиц планируется реализовать на основе двухэтапного подхода: сначала YOLOv11n обнаруживает лица на кадре, затем InsightFace извлекает эмбеддинги и сравнивает их с базой зарегистрированных сотрудников. Порог схожести будет настраиваться в зависимости от требуемого уровня безопасности на конкретной точке доступа.

Заключение

В ходе работы было выполнено задание пробного демо-экзамена: спроектирована БД, реализован серверный REST API и клиентский веб-интерфейс, всё зафиксировано в GitHub.

По дипломному проекту:

– проведено обучение модели YOLOv11m на датасете из 21 173 изображений с достижением mAP@0.5 = 98,6 %;

– реализован и протестирован модуль распознавания символов с нормализацией кириллица/латиница;

– разработан и протестирован полный пайплайн обработки видеокадра;

– разработана схема базы данных из 9 таблиц с поддержкой RBAC и журналирования событий.

Таким образом, в ходе учебной практики закреплены навыки применения современных инструментальных средств разработки, включая системы машинного обучения, инструменты работы с базами данных и системы контроля версий. Сформирована практическая база для завершения дипломного проекта.

Библиография

1 Документация Ultralytics YOLOv11 [Электронный ресурс] – Режим доступа: https://docs.ultralytics.com/.

2 Документация EasyOCR [Электронный ресурс] – Режим доступа: https://github.com/JaidedAI/EasyOCR.

3 Документация PostgreSQL 15 [Электронный ресурс] – Режим доступа: https://www.postgresql.org/docs/15/.

4 Документация платформы Tauri [Электронный ресурс] – Режим доступа: https://tauri.app/docs/.

5 Roboflow Universe. Датасет license-plate-recognition-rxg4e [Электронный ресурс] – Режим доступа: https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rxg4e.

6 Язык программирования Rust. Официальная документация [Электронный ресурс] – Режим доступа: https://doc.rust-lang.org/book/.

7 OpenCV. Документация по компьютерному зрению [Электронный ресурс] – Режим доступа: https://docs.opencv.org/.

8 Google Colaboratory. Руководство пользователя [Электронный ресурс] – Режим доступа: https://colab.research.google.com/notebooks/intro.ipynb.

9 InsightFace. Библиотека распознавания лиц [Электронный ресурс] – Режим доступа: https://github.com/deepinsight/insightface.

10 Федеральный закон №152-ФЗ «О персональных данных» [Электронный ресурс] – Режим доступа: http://www.consultant.ru/document/cons_doc_LAW_61801/.

11 sqlx. Асинхронная библиотека для Rust [Электронный ресурс] – Режим доступа: https://github.com/launchbadge/sqlx.

Приложение А
(справочное)
Листинг программы

Листинг А.1 – Полная схема БД системы АвтоТранс

```
CREATE TABLE "Users" (
    "UserID"   SERIAL        PRIMARY KEY,
    "Fio"      VARCHAR(255)  NOT NULL,
    "Phone"    VARCHAR(20)   NOT NULL,
    "Login"    VARCHAR(100)  UNIQUE NOT NULL,
    "Password" VARCHAR(255)  NOT NULL,
    "Type"     VARCHAR(50)   NOT NULL
);
CREATE TABLE "Requests" (
    "RequestID"          SERIAL        PRIMARY KEY,
    "StartDate"          DATE          NOT NULL,
    "CarType"            VARCHAR(100)  NOT NULL,
    "CarModel"           VARCHAR(100)  NOT NULL,
    "ProblemDescryption" TEXT          NOT NULL,
    "RequestStatus"      VARCHAR(50)   NOT NULL,
    "CompletionDate"     DATE,
    "RepairParts"        TEXT,
    "MasterID"  INTEGER
        REFERENCES "Users"("UserID") ON DELETE SET NULL,
    "ClientID"  INTEGER NOT NULL
        REFERENCES "Users"("UserID") ON DELETE CASCADE
);
CREATE TABLE "Comments" (
    "CommentID" SERIAL  PRIMARY KEY,
    "Message"   TEXT    NOT NULL,
    "MasterID"  INTEGER NOT NULL
        REFERENCES "Users"("UserID")       ON DELETE CASCADE,
    "RequestID" INTEGER NOT NULL
        REFERENCES "Requests"("RequestID") ON DELETE CASCADE
);
CREATE TABLE "Messages" (
    "MessageID"   SERIAL        PRIMARY KEY,
    "Subject"     VARCHAR(255)  NOT NULL,
    "MessageText" TEXT          NOT NULL,
    "ClientID"    INTEGER NOT NULL
        REFERENCES "Users"("UserID") ON DELETE CASCADE,
    "Reply"       TEXT,
    "IsRead"      BOOLEAN NOT NULL DEFAULT FALSE,
    "SentAt"      DATE
);

SELECT setval('"Users_UserID_seq"',
    (SELECT MAX("UserID") FROM "Users"));
SELECT setval('"Requests_RequestID_seq"',
    (SELECT MAX("RequestID") FROM "Requests"));
```

Листинг А.2 – Роли PostgreSQL и разграничение доступа

```
CREATE ROLE client_role;  CREATE ROLE mechanic_role;
CREATE ROLE manager_role; CREATE ROLE admin_role;

GRANT SELECT ON "Requests","Users","Comments","Messages"
    TO client_role;
GRANT INSERT ON "Requests","Messages" TO client_role;

GRANT SELECT, UPDATE ON "Requests","Comments" TO mechanic_role;
GRANT INSERT         ON "Comments"            TO mechanic_role;

GRANT SELECT, INSERT, UPDATE
    ON "Requests","Users","Comments","Messages"
    TO manager_role;

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public
    TO admin_role;
```

Листинг А.3 – Контроллер заявок

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

[ApiController]
[Route("api/[controller]")]
public class RequestsController : ControllerBase
{
    private readonly AppDbContext _db;
    public RequestsController(AppDbContext db) => _db = db;

    [Authorize] [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var type   = User.FindFirst("Type")?.Value;
        var  userId  =  int.Parse(User.FindFirst("UserID")?.Value  ??
"0");
        var q = _db.Requests
            .Include(r => r.Master).Include(r => r.Client)
            .AsQueryable();
        if (type == "заказчик")
            q = q.Where(r => r.ClientID == userId);
        else if (type == "автомеханик")
            q = q.Where(r => r.MasterID == userId);
        return Ok(await q.OrderByDescending(r => r.StartDate)
                         .ToListAsync());
    }

    [Authorize] [HttpGet("{id}")]
    public async Task<IActionResult> GetById(int id)
    {
        var r = await _db.Requests
            .Include(x => x.Master).Include(x => x.Client)
            .Include(x => x.Comments)
            .FirstOrDefaultAsync(x => x.RequestID == id);
```

```
        return r == null ? NotFound() : Ok(r);
    }

    [Authorize(Roles = "заказчик")] [HttpPost]
    public async Task<IActionResult> Create([FromBody] Request dto)
    {
        dto.ClientID                                                =
int.Parse(User.FindFirst("UserID")!.Value);
        dto.StartDate     = DateOnly.FromDateTime(DateTime.Today);
        dto.RequestStatus = "Новая";
        _db.Requests.Add(dto);
        await _db.SaveChangesAsync();
        return CreatedAtAction(nameof(GetById),
            new { id = dto.RequestID }, dto);
    }

    [Authorize] [HttpPut("{id}")]
    public async Task<IActionResult> Update(int id,
        [FromBody] Request dto)
    {
        var existing = await _db.Requests.FindAsync(id);
        if (existing == null) return NotFound();
        var type = User.FindFirst("Type")?.Value;
        if (type == "автомеханик")
        {
            existing.RequestStatus = dto.RequestStatus;
            if (dto.RequestStatus == "завершена")
                existing.CompletionDate =
                    DateOnly.FromDateTime(DateTime.Today);
            existing.RepairParts = dto.RepairParts;
        }
        else
        {
            existing.RequestStatus  = dto.RequestStatus;
            existing.MasterID       = dto.MasterID;
            existing.CompletionDate = dto.CompletionDate;
        }
        await _db.SaveChangesAsync();
        return NoContent();
    }

    [Authorize(Roles = "администратор")] [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        var r = await _db.Requests.FindAsync(id);
        if (r == null) return NotFound();
        _db.Requests.Remove(r); await _db.SaveChangesAsync();
        return NoContent();
    }
}
```

Листинг А.4 – Контроллер аутентификации

```
[ApiController]
```

```
     [Route("api/[controller]")]
     public class AuthController : ControllerBase
     {
         private readonly AppDbContext _db;
         public AuthController(AppDbContext db) => _db = db;

         [HttpPost("login")]
         public  async  Task<IActionResult>  Login([FromBody]  LoginDto
dto)
         {
             var user = await _db.Users
                 .FirstOrDefaultAsync(u => u.Login == dto.Login);
             if (user == null ||
                 !BCrypt.Net.BCrypt.Verify(dto.Password,
user.Password))
                 return Unauthorized("Неверный логин или пароль");
             var claims = new List<Claim> {
                 new("UserID", user.UserID.ToString()),
                 new("Fio",    user.Fio),
                 new("Type",   user.Type),
                 new(ClaimTypes.Role, user.Type)
             };
             await HttpContext.SignInAsync(
                 CookieAuthenticationDefaults.AuthenticationScheme,
                 new ClaimsPrincipal(new ClaimsIdentity(claims,

CookieAuthenticationDefaults.AuthenticationScheme)));
             return Ok(new { user.UserID, user.Fio, user.Type });
         }

         [Authorize] [HttpPost("logout")]
         public async Task<IActionResult> Logout()
         {
             await HttpContext.SignOutAsync(
                 CookieAuthenticationDefaults.AuthenticationScheme);
             return NoContent();
         }

         [HttpPost("register")]
         public    async    Task<IActionResult>    Register([FromBody]
RegisterDto dto)
         {
             if    (await    _db.Users.AnyAsync(u    =>    u.Login    ==
dto.Login))
                 return Conflict("Логин уже занят");
             _db.Users.Add(new User {
                 Fio      = dto.Fio,   Phone = dto.Phone,
                 Login    = dto.Login,
                 Password                                          =
BCrypt.Net.BCrypt.HashPassword(dto.Password),
                 Type     = "заказчик"
             });
             await _db.SaveChangesAsync();
```

```
            return StatusCode(201);
        }
    }
```

Листинг А.5 – Схема БД дипломного проекта

```
CREATE TABLE departments (id SERIAL PRIMARY KEY, name VARCHAR(100)
NOT NULL UNIQUE);

CREATE TABLE employees (
    id SERIAL PRIMARY KEY, full_name VARCHAR(255) NOT NULL,
    department_id INT REFERENCES departments(id),
    position VARCHAR(150), status VARCHAR(50),
    photo_path VARCHAR(500), comment TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TABLE users (
    id SERIAL PRIMARY KEY, employee_id INT REFERENCES employees(id),
    login VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_superadmin BOOLEAN NOT NULL DEFAULT FALSE,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TABLE vehicles (
    id SERIAL PRIMARY KEY, license_plate VARCHAR(20) NOT NULL
UNIQUE,
    owner_employee_id INT REFERENCES employees(id),
    pass_type VARCHAR(50) NOT NULL, pass_status VARCHAR(50) NOT
NULL,
    total_passes INT, remaining_passes INT,
    valid_from TIMESTAMP, valid_to TIMESTAMP, comment TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TABLE cameras (
    id SERIAL PRIMARY KEY, name VARCHAR(100) NOT NULL,
    ip_address VARCHAR(50) NOT NULL, port INT NOT NULL,
    login VARCHAR(100), password VARCHAR(200),
    is_active BOOLEAN NOT NULL DEFAULT TRUE, comment TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TABLE actions_log (
    id SERIAL PRIMARY KEY,
    event_time TIMESTAMP NOT NULL DEFAULT NOW(),
    user_id INT REFERENCES users(id),
    employee_id INT REFERENCES employees(id),
    vehicle_id  INT REFERENCES vehicles(id),
    camera_id   INT REFERENCES cameras(id),
    action_type VARCHAR(100) NOT NULL,
    status      VARCHAR(50)  NOT NULL, description TEXT
```

```
);
CREATE TABLE notifications (
    id SERIAL PRIMARY KEY,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    user_id INT REFERENCES users(id), text TEXT NOT NULL,
    level VARCHAR(20) DEFAULT 'info',
    is_read BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_emp_name ON employees(full_name);
CREATE INDEX idx_veh_plate ON vehicles(license_plate);
CREATE INDEX idx_log_time ON actions_log(event_time DESC);
CREATE INDEX idx_log_cam ON actions_log(camera_id);
CREATE INDEX idx_notif ON notifications(user_id, is_read);
```

Листинг А.6 – Пайплайн детекции и OCR номерных знаков

```
import cv2, easyocr, torch
from ultralytics import YOLO

MODEL_PATH     = "yolo11m.pt"
CONF_THRESHOLD = 0.45

model  = YOLO(MODEL_PATH)
reader = easyocr.Reader(["en", "ru"],
                        gpu=torch.cuda.is_available())

def detect_plate(frame):
    """Обнаружение номерного знака на кадре."""
    results     =     model.predict(frame,     conf=CONF_THRESHOLD,
verbose=False)
    return results[0].boxes.xyxy.cpu().numpy()

def estimate_contrast(roi):
    """Оценка контрастности ROI для выбора стратегии OCR."""
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    return gray.std()

def read_plate(frame, box):
    """OCR номерного знака по bounding box."""
    x1, y1, x2, y2 = map(int, box)
    roi = frame[y1:y2, x1:x2]
    roi = cv2.resize(roi, None, fx=2, fy=2,
                     interpolation=cv2.INTER_CUBIC)
    # Адаптивный выбор предобработки по контрасту
    if estimate_contrast(roi) < 40:
        gray    = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
        _, proc = cv2.threshold(gray, 0, 255,
                                cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    else:
        proc = roi
    result = reader.readtext(proc, detail=0)
    return " ".join(result).upper().strip()

def process_video(source=0):
```

```
    """Основной цикл обработки видеопотока."""
    cap = cv2.VideoCapture(source)
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret: break
        for box in detect_plate(frame):
            text = read_plate(frame, box)
            x1, y1, x2, y2 = map(int, box)
            cv2.rectangle(frame, (x1,y1), (x2,y2), (0,255,0), 2)
            cv2.putText(frame, text, (x1, y1-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8,
                        (0,255,0), 2)
            print(f"Detected: {text}")
        cv2.imshow("Vehicle Detection", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"): break
    cap.release(); cv2.destroyAllWindows()

if __name__ == "__main__":
    process_video(source="rtsp://camera_ip/stream")
```

Листинг А.7 – Модуль аутентификации клиентской части

```
document.getElementById('loginForm')
    .addEventListener('submit', function(e) {
    e.preventDefault();
    const login    = document.getElementById('login').value;
    const password = document.getElementById('password').value;
    document.getElementById('loadingOverlay').style.display          =
'flex';
    setTimeout(() => {
        if (login && password) {
            localStorage.setItem('isAuthenticated', 'true');
            localStorage.setItem('userLogin', login);
            window.location.href = 'dashboard.html';
        } else {
            alert('Введите логин и пароль');
            document.getElementById('loadingOverlay')
                .style.display = 'none';
        }
    }, 1500);
});
if (localStorage.getItem('isAuthenticated') === 'true')
    window.location.href = 'dashboard.html';
```

Листинг А.8 – Модуль журнала событий ДП (JavaScript)

```
document.addEventListener('DOMContentLoaded', () => {
    const log = document.getElementById('systemLogs');
    const refreshBtn = document.getElementById('refreshLogs');
    const clearBtn   = document.getElementById('clearLogs');
    let interval;
    function addEntry(time, type, message) {
        const el = document.createElement('div');
        el.className = 'log-entry';
```